A PROJECT REPORT ON

WindWatch AI

Wind Turbine Operating-Status Prediction using Machine Learning, deployed on AWS

Submitted by

Shrushti Malkar

Under the guidance of

Rithik Raj

TABLE OF CONTENTS

1. Abstract **1**

2. Introduction **2**

2.1 Context 2
2.2 Why predict the operating status 2
2.3 Challenges in the data 2
2.4 Overview of the solution 3
2.5 Organisation of the report 3

3. Background **4**

3.1 How a wind turbine operates 4
3.2 SCADA data 4
3.3 Operating status codes 4
3.4 Machine learning for condition monitoring 4
3.5 Class imbalance and evaluation 5
3.6 From a trained model to a web service 5

4. Aim **5**

5. Objectives **5**

6. Literature Review **7**

6.1 SCADA-based condition monitoring 7
6.2 Fault prediction and detection 7
6.3 Learning from imbalanced data 7
6.4 Tree ensembles and boosting 8
6.5 Interpretability 8
6.6 Public benchmark data 8

7. Literature Survey **9**

7.1 Summary of twelve studies 9
7.2 Observations from the survey 10
7.3 Research gap addressed by this project 10

8. Methodology **11**

8.1 Overall workflow 11
8.2 Data acquisition 11
8.3 Data cleaning 11
8.4 Data transformation 12
8.5 Exploratory data analysis 13
8.6 Models 13
8.7 Handling class imbalance 14
8.8 Evaluation metrics 14
8.9 Pipeline design and leakage control 14
8.10 Explaining a prediction 15
8.11 Web application design 15
8.12 Deployment on AWS 16
8.13 Tools and technologies 17

9. Code Screenshots **18**

9.1 Notebook: preparing the data 18
9.2 Notebook: exploration and modelling 20
9.3 Notebook: saving the model 21

9.4 Flask back end 22
9.5 Front end 24
9.6 Deployment files 24
10. Dataset Overview 26
10.1 Structure of the dataset 26
10.2 Class balance and the two turbines 27
10.3 Behaviour over time 27
10.4 The power curve 27
10.5 Sensors by status 28
10.6 Time of day 29
10.7 Correlations 29
10.8 What the exploration tells us 30
11. Input Screenshots 32
11.1 Landing page 32
11.2 Input mode 1: a single reading, sensor by sensor 32
11.3 Input mode 2: pasting a complete row 33
11.4 Input mode 3: uploading a dataset 34
11.5 Mobile layout 34
12. Output Screenshots and Results 36
12.1 Output of a single prediction 36
12.2 Output for a pasted row 37
12.3 Output for an uploaded dataset 38
12.4 Output through the API 39
12.5 Model comparison on the hold-out set 40
12.6 Per-class results 41
12.7 Overfitting check 42
12.8 Features that drive the model 42
12.9 Why Logistic Regression was selected 43
13. Conclusion 44
14. Summary 44
15. Future Scope 45
16. References 47
Appendix A: Project Folder Structure 48
Appendix B: Running and Deploying the Project 48
Appendix C: Glossary 49

LIST OF FIGURES

Figure 1: End-to-end workflow of the project....11
Figure 2: The prediction pipeline stored in the model file....15
Figure 3: System architecture and hosting options on AWS....15
Figure 4: Step 1: imports....18
Figure 5: Step 2: loading the Excel file....18
Figure 6: Step 3: data cleaning....19
Figure 7: Step 4: feature creation, correlation filter and train/test split....19
Figure 8: Step 5: code for the power curve and the box-plots....20
Figure 9: Step 6a: pipelines of the four models....20
Figure 10: Step 6b: training and evaluation loop....21
Figure 11: Step 7: selecting the best model and measuring feature importance....21
Figure 12: Step 8: saving the model bundle with joblib....22
Figure 13: application.py: cleaning incoming data and scoring....22
Figure 14: application.py: per-prediction explanation....23
Figure 15: application.py: the /api/predict endpoint....23
Figure 16: application.py: the start of the /api/predict_file endpoint....24
Figure 17: app.js: collecting input and parsing a pasted row....24
Figure 18: Procfile, nginx configuration and requirements....25
Figure 19: Dockerfile for container-based deployment....25
Figure 20: Class distribution, overall and per turbine....27
Figure 21: Monthly share of each status for the two turbines....27
Figure 22: Wind speed against power output, coloured by status....28
Figure 23: Box-plots of four key sensors by status....28
Figure 24: Density of wind speed and sensor_0_avg by status....28
Figure 25: Share of each status by hour of day....29
Figure 26: The 15 features most correlated with the status....30
Figure 27: Correlation heat-map of the 12 most relevant features....30
Figure 28: Landing page of WindWatch AI....32
Figure 29: Single reading: key sensors, sample selector and reset button....33
Figure 30: Single reading with all 61 features expanded....33
Figure 31: Paste-a-row input containing a complete JSON row....34
Figure 32: Dataset upload with the demo file selected....34
Figure 33: The landing page on a phone-sized screen....35
Figure 34: Prediction for a real record of status 0....36
Figure 35: Prediction for a real record of status 3....36
Figure 36: Prediction for a real record of status 5....37
Figure 37: Result for a complete pasted row....37
Figure 38: Result for a short CSV row (5 columns only)....38
Figure 39: Predictions for an uploaded dataset....39
Figure 40: Request and response of the /api/predict endpoint....40
Figure 41: Accuracy, macro-recall and macro-F1 of the four models....41
Figure 42: Confusion matrix of Logistic Regression (counts and row-normalised)....42
Figure 43: Top 15 features by permutation importance (Logistic Regression)....42

LIST OF TABLES

Table 1: Survey of twelve relevant studies.....9
Table 2: Data problems found and the remedy applied.....12
Table 3: Models and main settings.....13
Table 4: Endpoints of the web application.....16
Table 5: Technology stack.....17
Table 6: Records per turbine.....26
Table 7: Distribution of the target.....26
Table 8: Column groups of the raw file.....26
Table 9: Mean value of selected sensors by status.....29
Table 10: Hold-out results of the four models.....40
Table 11: Precision and recall of each model for each status.....41
Table 12: Files of the deliverable.....48
Table 13: Terms used in the report.....49

1. Abstract

Wind turbines report on their own condition every ten minutes. Each record carries wind speed, electrical power, temperatures, pressures and dozens of other readings, each summarised as an average, a minimum, a maximum or a standard deviation. Hidden in that stream is a label that matters a great deal to the people who run the machines: the operating status of the turbine at that moment. This project builds a machine-learning system that predicts that status from the sensor values alone, and packages the trained model as a web application that can be hosted on Amazon Web Services (AWS).

The work uses 160,637 ten-minute records from two turbines observed between April 2019 and May 2023, with 81 sensor statistics per record. The raw spreadsheet was not clean. It contained empty rows, a header line repeated in the middle of the data, and numeric columns that had been read as text because of that header. These faults were found and repaired, 21 near-duplicate sensors were removed, and hour-of-day and month features were added, leaving 61 model inputs. The target has three classes: status 0 (78.8 % of rows), status 5 (19.1 %) and status 3 (2.1 %). Because one class dominates, plain accuracy is misleading, and macro-averaged F1 was used to choose between models.

Four classifiers were trained with class weighting: Logistic Regression, Decision Tree, Random Forest and Histogram Gradient Boosting. They were tested on 6,505 records that the dataset reserves for prediction. The three tree-based models reached about 96 % accuracy, but they almost never recognised the rare status 3 (recall between 0 % and 1.4 %). Logistic Regression scored lower accuracy (86.2 %) but found 60.4 % of status 3 records and achieved the best macro-F1 (0.654) and macro-recall (0.818), with almost identical training and test scores. It was therefore selected, saved with joblib and served by a Flask application. The application accepts a single reading, a pasted row or an uploaded dataset, and returns class probabilities together with the sensors that pushed the decision.

Keywords: wind turbine, SCADA, operating status, classification, class imbalance, Logistic Regression, scikit-learn, Flask, joblib, AWS.

2. Introduction

2.1 Context

Wind power has become one of the main sources of new electricity capacity in many countries, and the machines that produce it are large, expensive and often remote. A single turbine may stand on a ridge or offshore, hours away from the nearest maintenance team. When it stops unexpectedly, the lost production is only part of the cost; a technician has to be sent, parts may have to be ordered, and the repair window depends on the weather. For this reason operators monitor their turbines continuously instead of waiting for visible failure.

The main source of that monitoring information is the SCADA system (Supervisory Control and Data Acquisition) installed on almost every commercial turbine. SCADA logs a standard set of signals at a fixed interval, usually ten minutes, and stores them in a central database. The signals are not specialised condition-monitoring measurements such as high-frequency vibration, but they are cheap, always available and cover the whole machine, which is why a large body of research treats them as the practical starting point for data-driven maintenance [1], [2].

2.2 Why predict the operating status

Every SCADA record also carries a status code, a small integer that tells the operator what the turbine was doing during that interval: producing power normally, stopped for service, or stopped or limited for some other reason. The code is assigned by the turbine controller, but looking it up is not always possible. Records may arrive without it, codes from different manufacturers are not always comparable, and analysts often want to know whether the *sensor pattern* of a record agrees with the code that was logged.

A model that predicts the status from the sensors answers that question. It can act as a consistency check on logged codes, as a way of labelling data that arrives unlabelled, and as a first step towards more demanding tasks such as early fault detection. It also forces the data to be examined properly, which in this project exposed several quality problems that would have silently damaged any model trained on the raw file.

2.3 Challenges in the data

Three difficulties shape the whole project.

- **Dirty input.** The Excel file joins the records of two turbines, and at the join a second header row was left inside the data. Because of it, every numeric column was loaded as text. Two completely empty rows and two single missing sensor values were also present.

- **Heavy class imbalance.** Status 0 accounts for 78.8 % of all records, status 5 for 19.1 % and status 3 for only 2.1 %. A model that always answered "status 0" would be right almost four times in five and useless in practice.
- **Redundant sensors.** Many of the 81 columns carry nearly the same information, for example the average, maximum and minimum of one physical quantity. Keeping all of them inflates the model and makes the coefficients hard to interpret.

2.4 Overview of the solution

The project follows the full life cycle of a machine-learning application. The raw Excel file is loaded and cleaned, new features are created, and eight exploratory charts are produced to understand how sensors behave under each status. Four algorithms are compared on a hold-out set using metrics that respect the class imbalance. The best pipeline is stored in one joblib file, and a Flask web application loads that file and offers three ways to obtain predictions: editing the sensors of a single reading one by one, pasting a complete row, or uploading an entire dataset. Deployment files for AWS Elastic Beanstalk, EC2 and App Runner complete the package.

2.5 Organisation of the report

Chapter 3 gives the technical background and Chapters 4 and 5 state the aim and objectives. Chapters 6 and 7 review the literature and survey twelve relevant studies. Chapter 8 explains the methodology, Chapter 9 shows the code, and Chapter 10 describes the dataset with the exploratory charts. Chapters 11 and 12 present screenshots of the inputs and outputs of the application together with the model results. Chapters 13 to 15 contain the conclusion, a summary and the future scope, followed by the references and appendices.

3. Background

3.1 How a wind turbine operates

A turbine starts producing electricity when the wind exceeds a minimum speed, usually called the cut-in speed. As the wind increases, power rises steeply until the machine reaches its rated output, after which the blades are pitched to hold the power constant. At very high wind speeds the turbine is shut down to protect it. Plotting power against wind speed gives the familiar S-shaped power curve, and deviations from this curve are a classical sign that something is wrong. The scatter plot in Chapter 10 reproduces the curve from the project data, with power normalised between 0 and 1.

This physical behaviour explains why wind speed and power are among the strongest signals for status prediction: at very low wind there is simply nothing to produce, and the turbine is expected to be standing still.

3.2 SCADA data

SCADA data is collected at ten-minute intervals. For each signal the system usually stores four statistics of the interval: the average, the minimum, the maximum and the standard deviation. In this dataset 46 columns are averages, 9 each are minimum, maximum and standard-deviation columns, and 8 are stored without a suffix, for a total of 81 sensor columns. The names are anonymised (`sensor_0_avg`, `sensor_18_avg` and so on), but a few can be recognised from their names, for example `wind_speed_3_avg`, `power_29_avg` and the `reactive_power_*` group.

The median spacing between consecutive records of a turbine is exactly 10 minutes; only 71 gaps are longer, which indicates a nearly complete series.

3.3 Operating status codes

The target column `status_type_id` takes the values 0, 3 and 5 in this dataset. Under the common SCADA convention, 0 denotes normal operation, 3 service or maintenance, and 5 another kind of downtime or limitation. The anonymised file does not include a data dictionary, so these meanings are an assumption that should be confirmed with the data owner; the model itself does not depend on them, and they appear only as display labels in the web application.

3.4 Machine learning for condition monitoring

Research on wind-turbine SCADA data uses two main families of methods. In the first, a model learns the *normal behaviour* of a signal, for example gearbox temperature as a function of power and ambient temperature, and large deviations are flagged as anomalies.

In the second, a *supervised classifier* learns from records that carry a label, such as a fault category or a status code [2]. This project belongs to the second family. A classifier is given a vector of sensor values **x** and returns one of three classes together with a probability for each.

3.5 Class imbalance and evaluation

When one class dominates, a classifier can obtain a high accuracy by ignoring the others. Two remedies are common: changing the learning problem, for instance by giving the rare class a larger weight in the loss function, and changing the data, for example by oversampling the rare class [7], [8]. The evaluation must also change. Precision, recall and F1 are computed for each class, and their unweighted mean (macro average) gives every class the same importance regardless of its size [11]. This project uses class weights and macro-F1; oversampling was left for future work because it would have increased the training time and the risk of leaking synthetic examples into the evaluation.

3.6 From a trained model to a web service

A trained model exists only in the memory of the notebook that created it. To reuse it, the fitted object is serialised to disk; the scikit-learn project recommends joblib for this purpose because it stores large NumPy arrays efficiently [13]. A web application then loads the file once at start-up and exposes the model through HTTP endpoints. In Python this is normally done with Flask, a lightweight web framework [17], run by a production-grade WSGI server such as Gunicorn. Finally the application is placed on a cloud platform. AWS offers several ways to do this: Elastic Beanstalk, which creates and manages the servers from the application folder; EC2, a plain virtual machine that gives full control; and App Runner, which runs a Docker container and provides an HTTPS address automatically.

4. Aim

The aim of this project is to design, implement and deploy an end-to-end machine-learning system that predicts the operating status of a wind turbine from its ten-minute SCADA sensor readings, and to make the resulting model usable by non-programmers through a web application hosted on AWS.

5. Objectives

The aim was broken down into the following measurable objectives.

- **O1 – Data preparation.** Load the raw Excel dataset, identify every structural problem (empty rows, repeated header, text-typed numbers, constant columns, duplicates, missing values) and produce a clean table without losing valid records.
- **O2 – Feature preparation.** Create time-based features, encode the target, remove redundant sensors using training data only, and split the data with the train/prediction flag supplied in the file so that no test information leaks into training.
- **O3 – Exploratory analysis.** Produce at least eight charts that show the class balance, the behaviour over time, the power curve and the relationship between key sensors and each status.
- **O4 – Model comparison.** Train at least four different algorithms with a consistent pipeline and compare them with accuracy, macro-precision, macro-recall, macro-F1 and weighted-F1.
- **O5 – Model selection.** Select the best model with a justified criterion, examine its errors with a confusion matrix and per-class scores, and identify the sensors that drive its decisions.
- **O6 – Model packaging.** Save the whole pipeline and the metadata needed by an application (feature names, typical values, ranges, class labels, metrics, sample rows) in one joblib file.
- **O7 – Web application.** Build a web interface that accepts a single reading edited sensor by sensor, a pasted full row, or an uploaded CSV/Excel dataset, and returns the predicted status, the probability of each class and an explanation.
- **O8 – Deployment.** Prepare and document deployment on AWS using Elastic Beanstalk, EC2 or App Runner, including a health-check endpoint.

6. Literature Review

This chapter groups the literature that shaped the project by theme. Chapter 7 then summarises twelve individual studies in a table.

6.1 SCADA-based condition monitoring

Tautz-Weinert and Watson reviewed the use of SCADA data for condition monitoring and sorted the published methods into a handful of families: trending of single signals, clustering, normal-behaviour modelling, damage modelling and the assessment of alarms by expert systems [1]. Their review is useful here for two reasons. It confirms that ten-minute SCADA records, although coarse, contain enough information to support useful models, and it shows that most published work concentrates on detecting anomalies in particular components rather than on predicting the general operating state of the machine.

Stetco and co-authors later surveyed machine-learning techniques for turbine condition monitoring, covering supervised and unsupervised methods applied to gearboxes, generators, blades and other components [2]. The review documents how widely neural networks, support-vector machines, decision trees and ensembles have been used, and it is a good map of the methods that a student project can reasonably compare.

6.2 Fault prediction and detection

Several authors turned SCADA signals into predictions about faults. Kusiak and Li applied data-mining algorithms to SCADA and fault-log data in order to predict fault categories and specific faults at different time horizons [3]. Zaher and colleagues proposed an automated, online analysis of SCADA data that learns the normal behaviour of the signals and raises alarms when the behaviour changes [4]. Schlechtingen and Santos compared neural-network-based and regression-based normal-behaviour models on real SCADA data and discussed how each approach copes with the non-linear relations between signals [5]. Together these papers show the two routes mentioned in Chapter 3: learning normal behaviour, or learning from labelled states. Their common message for this project is that the choice of model matters less than the quality of the data and of the evaluation.

6.3 Learning from imbalanced data

Fault and status data are almost always imbalanced. He and Garcia reviewed the field of imbalanced learning and grouped the remedies into sampling methods, cost-sensitive learning and appropriate assessment metrics [7]. Chawla and co-authors introduced SMOTE, which creates synthetic examples of the minority class by interpolating between neighbours instead of merely duplicating records [8]. Sokolova and Lapalme analysed a large set of classification measures and how they respond to changes in the class

distribution, which is the reason why accuracy is replaced by macro-averaged measures when classes differ greatly in size [11]. This project adopts the cost-sensitive route (class weights) and the macro metrics, and leaves resampling for future work.

6.4 Tree ensembles and boosting

Breiman's Random Forest trains many decision trees on random subsets of records and features and averages their votes, which reduces the variance of a single tree [9]. Gradient boosting builds trees sequentially so that each tree corrects the errors of the previous ones; XGBoost made this approach fast and scalable [10], and LightGBM introduced histogram-based learning that scales to large tables [16]. The Histogram Gradient Boosting classifier in scikit-learn follows the same histogram idea [13]. These ensembles are the usual first choice for tabular data and therefore form the strongest competition for the simple linear model that was finally selected.

6.5 Interpretability

For maintenance staff a prediction is more useful when it is accompanied by a reason. Lundberg and Lee unified several explanation methods under the SHAP framework, which assigns each feature a contribution to an individual prediction [12]. A linear model offers a simple, exact form of the same idea: the contribution of a feature to the score of a class is its coefficient multiplied by its standardised value [14], [15]. The web application of this project uses this simple form to show the six sensors that influenced a prediction most.

6.6 Public benchmark data

Real SCADA datasets with trustworthy labels are scarce. Gück, Roelofs and Faulstich published a multi-farm benchmark of real turbine data with labelled events and a proposed evaluation scheme, aiming at fairer comparison of anomaly-detection methods [6]. Their work highlights the need to evaluate on realistic data and to be cautious with results obtained on small or artificially balanced sets, an advice that the hold-out design of the present project tries to follow.

7. Literature Survey

7.1 Summary of twelve studies

Table 1 lists the studies in the order in which they appear in Chapter 6, together with their focus and their relevance to this project.

Table 1: Survey of twelve relevant studies

No.	Authors (year)	Focus	Approach	Relevance to this project
1	Tautz-Weinert and Watson (2017) [1]	Review of SCADA-based condition monitoring	Groups methods into trending, clustering, normal-behaviour models, damage models and expert-system alarm assessment	Confirms SCADA as a practical data source and frames this work as data-driven monitoring
2	Stetco et al. (2019) [2]	Review of machine learning for turbine condition monitoring	Surveys supervised and unsupervised methods applied to turbine components	Gives the taxonomy used to position the classifiers compared here
3	Kusiak and Li (2011) [3]	Prediction and diagnosis of turbine faults	Data-mining algorithms on SCADA and fault-log data, predicting fault categories at several horizons	Example of supervised classification of discrete states from SCADA
4	Zaher et al. (2009) [4]	Online fault detection	Automated analysis of SCADA signals using learned normal behaviour and alarms	Shows the anomaly-detection route that this project does not take
5	Schlechtingen and Santos (2011) [5]	Normal-behaviour modelling	Compares neural-network and regression-based models on real SCADA data	Motivates comparing a linear model with non-linear ones
6	Gück, Roelofs and Faulstich (2024) [6]	Benchmark dataset	Real multi-farm turbine data with labelled events and an evaluation scheme	Reference for future work on public data and realistic evaluation
7	He and Garcia (2009) [7]	Learning from imbalanced data	Reviews sampling, cost-sensitive learning and assessment metrics	Justifies class weighting and macro-averaged metrics
8	Chawla et al. (2002) [8]	SMOTE	Synthetic minority over-sampling by interpolation between neighbours	Candidate technique to improve the rare status 3
9	Breiman (2001) [9]	Random Forests	Ensemble of randomised decision trees combined by voting	One of the four models compared
10	Chen and Guestrin (2016) [10]	XGBoost	Scalable gradient tree boosting	Representative of the boosting family; Histogram Gradient Boosting follows a similar idea

No.	Authors (year)	Focus	Approach	Relevance to this project
11	Sokolova and Lapalme (2009) [11]	Classification performance measures	Analyses how measures behave when class balance changes	Supports macro-averaged precision, recall and F1
12	Lundberg and Lee (2017) [12]	Explaining predictions	SHAP, additive per-feature contributions	Inspires the "why this prediction" panel and future SHAP integration

7.2 Observations from the survey

Reading the studies side by side leads to four observations that influenced the design of this project.

- Most work on turbine SCADA data targets **component faults** or anomalies, and uses normal-behaviour models. Predicting the general **operating status** with a supervised classifier is less common and therefore a reasonable, well-defined task for a capstone project.
- Imbalanced classes are the rule, not the exception. The literature is consistent that accuracy must be complemented by class-wise or macro-averaged measures [7], [11].
- Tree ensembles are the default recommendation for tabular data [9], [10], [16], but they are not guaranteed to win. Whether they do depends on how the rare classes are handled and on whether the test data resembles the training data.
- Explanations matter to users. Even a simple, exact explanation of a linear model can make a prediction easier to trust than a bare class label [12].

7.3 Research gap addressed by this project

The surveyed work is mostly published as papers and experiments. Far fewer projects take the extra step of putting a trained model in front of an operator. This project therefore adds three things to the picture: a documented repair of a real, messy SCADA export; an honest model comparison in which the winner is chosen by a metric that suits the imbalanced target and not by accuracy alone; and a deployable web application that accepts data in three convenient ways and explains each prediction.

8. Methodology

8.1 Overall workflow

The project follows the sequence shown in Figure 1. Each stage produces a defined output that the next stage consumes: the raw spreadsheet becomes a clean table, the clean table becomes a feature matrix, the feature matrix is used to train and compare models, and the winning pipeline is stored in a file that the web application loads. All the data-science steps live in a single Jupyter notebook so that the whole analysis can be repeated by running the notebook from top to bottom.

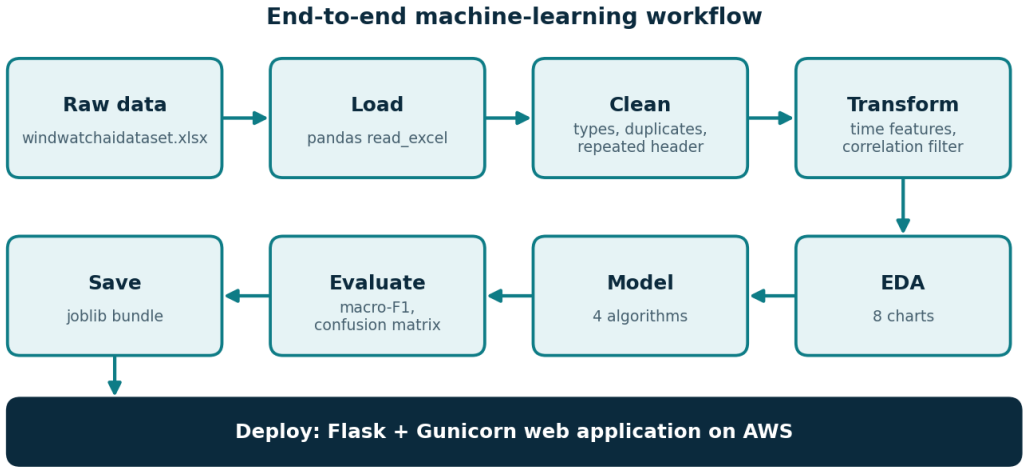


Figure 1: End-to-end workflow of the project

8.2 Data acquisition

The data was supplied as one Excel workbook, `windwatchaidataset.xlsx` (about 82 MB). It holds 160,640 rows and 86 columns: a timestamp, the turbine identifier (`asset_id`), a record `id`, a flag `train_test` that marks each row as *train* or *prediction*, the target `status_type_id`, and 81 sensor columns. Reading the file with pandas takes roughly one minute because Excel is a slow format for tables of this size.

8.3 Data cleaning

Inspection of the loaded table revealed the problems listed in Table 2. The most important one was not visible at first glance: when two per-turbine files were concatenated, the header line of the second file stayed in the data. Because that row contains text in every column, pandas stored all numeric columns as text (`object`), which would have made every statistical operation fail or, worse, silently treat numbers as strings.

Table 2: Data problems found and the remedy applied

Problem	How it was detected	Remedy
2 completely empty rows	all values missing	removed with `dropna(how="all")`
1 repeated header row	`time_stamp` equal to the text "time_stamp"	row removed
Numeric columns stored as text	`dtypes` showed `object`	`pd.to_numeric(errors="coerce")`
Timestamp stored as text	`dtypes`	`pd.to_datetime`
2 constant columns	`nunique() <= 1`	columns dropped
2 isolated missing sensor values	`isna().sum()`	median imputation inside the model pipeline
Possible duplicate records	duplicates on `asset_id` + `time_stamp`	none found (0)

After cleaning, 160,637 records remained; no valid record was discarded.

8.4 Data transformation

Time features. The hour of day and the month were extracted from the timestamp. The exploratory analysis in Chapter 10 shows that the share of status 5 changes during the day, so these features carry information.

Target encoding. The codes 0, 3 and 5 were mapped to 0, 1 and 2, as required by scikit-learn, and the mapping was stored so that predictions can be translated back to the original codes.

Removal of redundant sensors. The absolute correlation matrix of the training features was computed, and every feature that was correlated above 0.98 with an earlier feature was dropped. This removed 21 columns. The decision was made on training data only. After the removal 61 features remained, including `asset_id`, `hour` and `month`.

Train and test split. The file itself marks 154,132 rows as *train* and 6,505 rows as *prediction*. The *prediction* rows also carry the true status, so they serve as an independent hold-out set. Using the split supplied with the data avoids the optimistic results that a random split of overlapping time series would give.

Imputation and scaling. The two missing sensor values are replaced by the training median, and, for the linear model, every feature is standardised: $z = (x - \mu) / \sigma$. Both operations are placed inside the model pipeline, so that their parameters are learned from the training data only.

8.5 Exploratory data analysis

Eight charts were drawn with Matplotlib and Seaborn: the class distribution overall and per turbine, the monthly status share over time, the power curve coloured by status, box-plots and density plots of four key sensors, the status share by hour of day, the features most correlated with the target, and a correlation heat-map. They are discussed in Chapter 10. Their purpose was to find out whether the three statuses can be separated by the sensors at all, and to detect surprises such as a status that occurs on only one turbine.

8.6 Models

Four classifiers of increasing complexity were compared. Their settings are summarised in Table 3.

Logistic Regression is a linear model. For a record with standardised features **z** it computes a score for every class and turns the scores into probabilities with the softmax function:

$$P(y = k \mid z) = \exp(w_k \cdot z + b_k) / \Sigma_j \exp(w_j \cdot z + b_j)$$

The weights are learned by minimising the regularised log-loss. The model is fast, stable and its coefficients can be read directly [14], [15].

Decision Tree splits the data repeatedly on one feature at a time, choosing the split that most reduces the Gini impurity of the resulting groups:

$$G = 1 - \Sigma_k p_k^2$$

Depth was limited to 12 and each leaf required at least 5 records to restrain overfitting.

Random Forest trains 150 trees on bootstrap samples with random subsets of features and averages their votes [9]. It reduces the variance of single trees but, with 150 deep trees, can still memorise the training set.

Histogram Gradient Boosting builds 200 shallow trees one after another, each fitted to the errors of the ensemble so far, and bins the feature values into histograms to make the computation fast on 150,000 rows [10], [16].

Table 3: Models and main settings

Model	Main settings	Scaling needed
Logistic Regression	max_iter = 300, class_weight = balanced	yes (StandardScaler)
Decision Tree	max_depth = 12, min_samples_leaf = 5, class_weight = balanced	no
Random Forest	150 trees, min_samples_leaf = 2, class_weight = balanced_subsample	no
Histogram Gradient Boosting	200 iterations, learning_rate = 0.1, class_weight = balanced	no

All four models use the same random seed (42), the same imputer and the same training data, so that differences in results come from the algorithms and not from the set-up.

8.7 Handling class imbalance

The classes were balanced by weighting, not by resampling. With the setting *balanced*, each record is weighted by

$$w_c = n / (K \cdot n_c)$$

where n is the number of training records, K the number of classes (three) and n_c the number of records of class c. With the 154,132 training records of this project the weights come out at about 0.43 for status 0, 1.71 for status 5 and 15.97 for status 3, so a mistake on a record of the rare status 3 costs the model roughly 38 times more than a mistake on a record of status 0. This is the cost-sensitive approach discussed in [7].

8.8 Evaluation metrics

For each class k, the precision, recall and F1 score are

$$P_k = TP_k / (TP_k + FP_k) \qquad R_k = TP_k / (TP_k + FN_k) \qquad F1_k = 2 \cdot P_k \cdot R_k / (P_k + R_k)$$

and the macro averages are the unweighted means of the three class values, for example macro-F1 = (F1_0 + F1_3 + F1_5) / 3. The weighted F1 uses the class sizes as weights and therefore behaves like accuracy when one class dominates. The accuracy and weighted-F1 are reported for completeness, but **macro-F1 on the hold-out set is the selection criterion**, with macro-recall as the second consideration because missing an abnormal state is the costlier error in monitoring. The difference between the training and the hold-out score is used as a simple overfitting check.

8.9 Pipeline design and leakage control

Every model is a scikit-learn `Pipeline` made of an imputer, an optional scaler and the classifier (Figure 2). Because the steps are fitted together on the training data only, the medians and standardisation parameters never see the hold-out records. The same pipeline object is what is stored in the joblib file, which guarantees that the web application applies exactly the same transformations as the notebook.

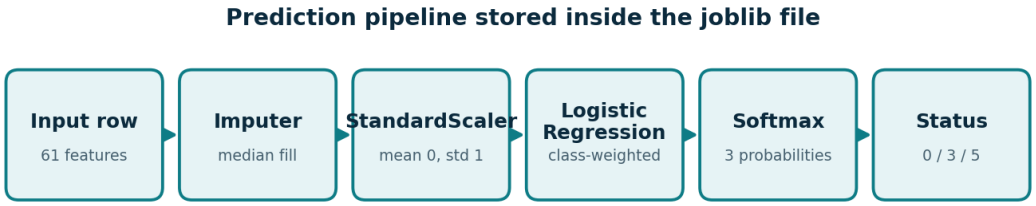


Figure 2: The prediction pipeline stored in the model file

8.10 Explaining a prediction

For the selected linear model, the contribution of feature j to the score of the predicted class k is the standardised value of the feature multiplied by the coefficient of that feature for the class:

$$c_j = z_j \cdot w_(k, j)$$

The application sorts the contributions by absolute size and shows the six largest, with a bar that points towards the predicted class when the contribution is positive and away from it when it is negative. In the notebook, the global importance of each feature is measured with permutation importance: the macro-F1 on a random sample of 4,000 hold-out records is recomputed after shuffling one feature, and the drop in the score is the importance of that feature.

8.11 Web application design

The application (Figure 3) is made of three parts. The **front end** is a single HTML page with CSS and JavaScript and no build step. The **back end** is a Flask application, `application.py`, run by Gunicorn. The **model file** is loaded once when the server starts. The back end exposes the endpoints listed in Table 4.

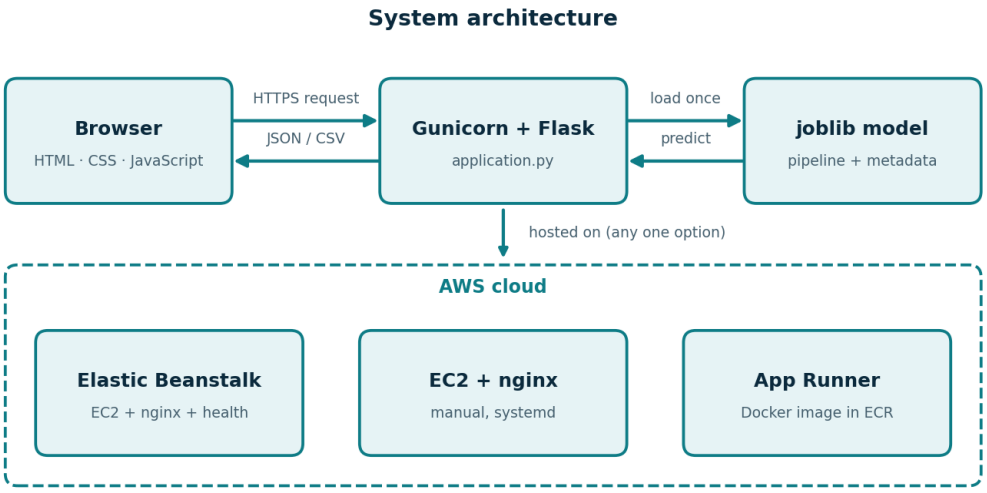


Figure 3: System architecture and hosting options on AWS

Table 4: Endpoints of the web application

Method	URL	Purpose
GET	/	Serves the web page
GET	/health	Returns the model name; used by AWS health checks
GET	/api/schema	Features with typical values and ranges, class labels, metrics, sample rows
POST	/api/predict	One row as JSON; returns class, probabilities, top factors
POST	/api/predict_file	CSV or Excel file; returns per-row predictions and a summary
GET	/sample	Downloads a 300-row demo file

The input handling follows one principle: **whatever the user does not supply is filled with a typical value**. The application applies the same cleaning rules as the notebook (drop empty rows and repeated headers, convert to numbers, derive hour and month from the timestamp), aligns the columns with the 61 model features and replaces every missing column by its training median. If an uploaded file contains a `status_type_id` column, the accuracy and macro-F1 of the predictions are computed and displayed as well. An upload is limited to 40 MB and to the first 50,000 rows to keep response times short.

8.12 Deployment on AWS

Three deployment routes were prepared and documented, so that the project can be hosted according to the available budget and time.

- **Elastic Beanstalk** needs only the project folder: `Procfile` tells it to start Gunicorn, and a small nginx configuration raises the upload limit from the default 1 MB to 50 MB, which the file-upload feature requires.
- **EC2** gives full control: Gunicorn is kept alive by a systemd service and nginx acts as the reverse proxy.
- **App Runner** runs the Docker image built from the supplied `Dockerfile` after it has been pushed to Amazon ECR, and provides an HTTPS address.

In all three cases the `/health` endpoint lets the platform verify that the application is running.

8.13 Tools and technologies

Table 5: Technology stack

Area	Tools
Language and notebook	Python 3.12, Jupyter
Data handling	pandas, NumPy, openpyxl
Visualisation	Matplotlib, Seaborn
Machine learning	scikit-learn 1.8 (Pipeline, LogisticRegression, DecisionTree, RandomForest, HistGradientBoosting), joblib
Back end	Flask, Gunicorn
Front end	HTML5, CSS3, JavaScript (no framework)
Cloud and packaging	AWS Elastic Beanstalk, EC2, App Runner, ECR, Docker, nginx

9. Code Screenshots

This chapter shows the central parts of the implementation as screenshots of the actual source code. The notebook cells appear in the order in which they run.

9.1 Notebook: preparing the data

The notebook starts with the imports (Figure 4). They show the structure of the solution at a glance: pandas and NumPy for data, Matplotlib and Seaborn for charts, and a set of scikit-learn classes for the pipeline, the four models and the metrics.

notebook · Step 1 – Imports

```
import warnings
warnings.filterwarnings("ignore")

import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             classification_report, confusion_matrix, ConfusionMatrixDisplay)

sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.dpi"] = 100
RANDOM_STATE = 42
```

Figure 4: Step 1: imports

The data is read with `pd.read_excel` (Figure 5). The notebook looks for the file in two locations, so that it works both from the notebook folder and from the project's `data` folder, and prints the number of columns of each data type; the large number of `object` columns is the first hint that the file contains text where numbers are expected.

notebook · Step 2 – Data load

```
import os
candidates = ["windwatchaidataset.xlsx", "../data/windwatchaidataset.xlsx"]
FILE_PATH = next((p for p in candidates if os.path.exists(p)), candidates[0])   # change if the file is elsewhere
print("Reading:", FILE_PATH)

df_raw = pd.read_excel(FILE_PATH)       # ~160k rows, takes about a minute
print("Raw shape:", df_raw.shape)
df_raw.head()

# Quick look at column types — many numeric columns were read as 'object' (a sign of dirty rows)
print(df_raw.dtypes.value_counts())
print("\nColumn groups:")
print("  identifiers :", ["time_stamp", "asset_id", "id", "train_test"])
print("  target      : status_type_id")
print("  sensors     :", df_raw.shape[1] - 5)
```

Figure 5: Step 2: loading the Excel file

The cleaning cell (Figure 6) removes empty rows and the repeated header, converts the types, removes duplicates and drops constant columns. It prints how many rows it dropped, so that every change to the data is visible in the notebook output.

notebook · Step 3 – Data cleaning

```
df = df_raw.copy()

# 1) empty rows
n_empty = df.isna().all(axis=1).sum()
df = df.dropna(how="all")

# 2) repeated header row(s)
mask_header = df["time_stamp"].astype(str) == "time_stamp"
n_header = mask_header.sum()
df = df[~mask_header]
print(f"Dropped {n_empty} empty rows and {n_header} repeated header row(s)")

# 3) correct data types
df["time_stamp"] = pd.to_datetime(df["time_stamp"])
df["asset_id"] = df["asset_id"].astype(int)
df["status_type_id"] = df["status_type_id"].astype(int)
sensor_cols = [c for c in df.columns if c not in ["time_stamp", "asset_id", "id", "train_test", "status_type_id"]]
df[sensor_cols] = df[sensor_cols].apply(pd.to_numeric, errors="coerce")

# 4) duplicates
n_dup = df.duplicated(subset=["asset_id", "time_stamp"]).sum()
df = df.drop_duplicates(subset=["asset_id", "time_stamp"])
print("Duplicate asset/timestamp rows removed:", n_dup)

# 5) constant columns
const_cols = [c for c in sensor_cols if df[c].nunique(dropna=True) <= 1]
df = df.drop(columns=const_cols)
sensor_cols = [c for c in sensor_cols if c not in const_cols]
print("Constant columns dropped:", const_cols)

df = df.sort_values(["asset_id", "time_stamp"]).reset_index(drop=True)
print("\nClean shape:", df.shape)
```

Figure 6: Step 3: data cleaning

The transformation cell (Figure 7) creates the time features, encodes the target, splits the data using the `train_test` flag, drops highly correlated features and builds the matrices `X_train`, `X_test`, `y_train` and `y_test`.

notebook · Step 4 – Data transformation

```
# time features
df["hour"] = df["time_stamp"].dt.hour
df["month"] = df["time_stamp"].dt.month

# target encoding
status_map = {0: 0, 3: 1, 5: 2}
inv_status = {v: k for k, v in status_map.items()}
df["target"] = df["status_type_id"].map(status_map)
class_names = [f"status {inv_status[i]}" for i in range(3)]

feature_cols = sensor_cols + ["asset_id", "hour", "month"]

train_df = df[df["train_test"] == "train"]
test_df  = df[df["train_test"] == "prediction"]

# drop near-duplicate (highly correlated) features, decided on train only
corr = train_df[feature_cols].corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_corr = [c for c in upper.columns if (upper[c] > 0.98).any()]
feature_cols = [c for c in feature_cols if c not in drop_corr]
print(f"Dropped {len(drop_corr)} highly-correlated features; {len(feature_cols)} features remain")

X_train, y_train = train_df[feature_cols], train_df["target"]
X_test,  y_test  = test_df[feature_cols],  test_df["target"]
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("\nTrain class share:\n", y_train.value_counts(normalize=True).round(3).rename(index=lambda i: class_names[i]))
```

Figure 7: Step 4: feature creation, correlation filter and train/test split

9.2 Notebook: exploration and modelling

Two of the exploratory charts are produced by the short snippets in Figure 8: the power curve with a sample of 25,000 points, and the box-plots of four key sensors by status.

notebook · Step 5 – EDA (power curve and box-plots)

```
# Graph 3 – Power curve: wind speed vs power, coloured by status
s = df.sample(25000, random_state=RANDOM_STATE)
plt.figure(figsize=(9, 5))
sns.scatterplot(data=s, x="wind_speed_3_avg", y="power_29_avg", hue="status_type_id",
                palette="Set1", alpha=0.4, s=12)
plt.title("Wind speed vs power output by status"); plt.show()

# Graph 4 – Boxplots of key sensors by status
key = ["wind_speed_3_avg", "power_29_avg", "sensor_0_avg", "sensor_18_avg"]
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, c in zip(axes, key):
    sns.boxplot(data=df, x="status_type_id", y=c, order=order, ax=ax, showfliers=False)
    ax.set_title(c)
plt.tight_layout(); plt.show()
```

Figure 8: Step 5: code for the power curve and the box-plots

The four models are defined as pipelines in a dictionary (Figure 9). Keeping them in one structure allows a single loop to train and evaluate all of them with identical code.

notebook · Step 6 – Model definitions

```
models = {
    "Logistic Regression": Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
        ("clf", LogisticRegression(max_iter=300, class_weight="balanced", random_state=RANDOM_STATE))]),
    "Decision Tree": Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("clf", DecisionTreeClassifier(max_depth=12, min_samples_leaf=5, class_weight="balanced", random_state=RANDOM_STATE))]),
    "Random Forest": Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("clf", RandomForestClassifier(n_estimators=150, min_samples_leaf=2, class_weight="balanced_subsample",
                                       n_jobs=-1, random_state=RANDOM_STATE))]),
    "Hist Gradient Boosting": Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("clf", HistGradientBoostingClassifier(max_iter=200, learning_rate=0.1, class_weight="balanced",
                                               random_state=RANDOM_STATE))]),
}
```

Figure 9: Step 6a: pipelines of the four models

The loop in Figure 10 fits each pipeline, predicts on the training and hold-out sets, and collects accuracy, macro precision, macro recall, macro-F1, weighted-F1 and the fitting time in a results table.

notebook · Step 6 – Training and evaluation loop

```
results, fitted = [], {}
for name, pipe in models.items():
    t0 = time.time()
    pipe.fit(X_train, y_train)
    fit_time = time.time() - t0
    pred_tr, pred_te = pipe.predict(X_train), pipe.predict(X_test)
    results.append({
        "Model": name,
        "Train macro-F1": f1_score(y_train, pred_tr, average="macro"),
        "Test accuracy": accuracy_score(y_test, pred_te),
        "Test precision (macro)": precision_score(y_test, pred_te, average="macro"),
        "Test recall (macro)": recall_score(y_test, pred_te, average="macro"),
        "Test macro-F1": f1_score(y_test, pred_te, average="macro"),
        "Test weighted-F1": f1_score(y_test, pred_te, average="weighted"),
        "Fit time (s)": fit_time,
    })
    fitted[name] = pipe
    print(f"✔ {name} trained in {fit_time:.1f}s")

results_df = pd.DataFrame(results).set_index("Model").sort_values("Test macro-F1", ascending=False)
results_df.round(3)
```

Figure 10: Step 6b: training and evaluation loop

The best model is picked automatically as the first row of the results table sorted by hold-out macro-F1 (Figure 11). The cell then prints the class-wise classification report and computes permutation importance on a hold-out sample.

notebook · Step 7 – Best model, report and importance

```
best_name = results_df.index[0]
best_model = fitted[best_name]
print("🏆 Best model (highest test macro-F1):", best_name)
print()
y_pred = best_model.predict(X_test)
print(classification_report(y_test, y_pred, target_names=class_names, digits=3))

# Which features drive the best model? (permutation importance on a test sample)
samp = X_test.sample(min(4000, len(X_test)), random_state=RANDOM_STATE)
perm = permutation_importance(best_model, samp, y_test.loc[samp.index], scoring="f1_macro",
                              n_repeats=3, random_state=RANDOM_STATE, n_jobs=1)
imp = pd.Series(perm.importances_mean, index=feature_cols).sort_values(ascending=False).head(15)
plt.figure(figsize=(8, 5))
sns.barplot(x=imp.values, y=imp.index, color="seagreen")
plt.title(f"Top 15 features – {best_name}"); plt.xlabel("drop in macro-F1 when shuffled"); plt.show()
```

Figure 11: Step 7: selecting the best model and measuring feature importance

9.3 Notebook: saving the model

The last notebook step (Figure 12) writes the deployable bundle. In addition to the fitted pipeline, the dictionary stores the feature list, the training medians used as defaults, the value ranges, the class codes and labels, the comparison metrics, the importance scores and six real sample rows. The application reads everything it needs from this one file.

notebook · Step 8 – Saving the model bundle

```
import joblib

# Meaning of the status codes (standard SCADA coding — confirm against your data dictionary)
STATUS_LABELS = {0: "Normal Operation", 3: "Service / Maintenance", 5: "Other / Downtime"}

importance_all = pd.Series(perm.importances_mean, index=feature_cols).sort_values(ascending=False)

# a few real rows from the hold-out set, so the web app can offer "load sample row" buttons
samples = []
for code_, g in test_df.groupby("status_type_id"):
    for _, row in g.sample(2, random_state=RANDOM_STATE).iterrows():
        samples.append({"actual_status": int(code_), "values": {c: float(row[c]) for c in feature_cols}})

bundle = {
    "pipeline": best_model,                     # imputer + scaler + classifier
    "model_name": best_name,
    "features": feature_cols,
    "defaults": {c: float(v) for c, v in X_train.median().items()},
    "ranges": {c: [float(X_train[c].min()), float(X_train[c].max())] for c in feature_cols},
    "class_codes": [inv_status[i] for i in range(3)],      # model output index -> original status code
    "status_labels": STATUS_LABELS,
    "metrics": results_df.round(4).reset_index().to_dict("records"),
    "feature_importance": {k: float(v) for k, v in importance_all.items()},
    "samples": samples,
    "train_rows": int(len(X_train)), "test_rows": int(len(X_test)),
    "sklearn_version": __import__("sklearn").__version__,
}
os.makedirs("../model", exist_ok=True)
joblib.dump(bundle, "../model/wind_turbine_model.joblib", compress=3)
print("Saved ../model/wind_turbine_model.joblib  (", round(os.path.getsize("../model/wind_turbine_model.joblib")/1024), "KB )")

# a small demo file the web app can upload (raw column names, includes the real status for comparison)
os.makedirs("../sample_data", exist_ok=True)
parts = [test_df[test_df.status_type_id == 0].sample(120, random_state=1),
         test_df[test_df.status_type_id == 3].sample(60, random_state=1),
         test_df[test_df.status_type_id == 5].sample(120, random_state=1)]
demo = pd.concat(parts).sort_values(["asset_id", "time_stamp"])
demo[["time_stamp", "asset_id", "status_type_id"] + sensor_cols].to_csv("../sample_data/sample_upload.csv", index=False)
print("Saved ../sample_data/sample_upload.csv", demo.shape[0], "rows")
```

Figure 12: Step 8: saving the model bundle with joblib

9.4 Flask back end

The function `clean_frame` in Figure 13 repeats the notebook's cleaning rules for data that arrives through the web, and aligns the columns with the model features; `run_model` returns the class index and the probabilities.

application.py – cleaning, scoring and explanation

```
def clean_frame(raw: pd.DataFrame):
    """Same cleaning rules as the notebook, then align to the model's feature list."""
    df = raw.dropna(how="all").copy()
    if "time_stamp" in df.columns:                                  # repeated header rows
        df = df[df["time_stamp"].astype(str) != "time_stamp"]
    if "time_stamp" in df.columns:
        ts = pd.to_datetime(df["time_stamp"], errors="coerce")
        if "hour" not in df.columns:
            df["hour"] = ts.dt.hour
        if "month" not in df.columns:
            df["month"] = ts.dt.month
    present = [c for c in FEATURES if c in df.columns]
    missing = [c for c in FEATURES if c not in df.columns]
    X = pd.DataFrame(index=df.index)
    for c in FEATURES:
        X[c] = pd.to_numeric(df[c], errors="coerce") if c in present else DEFAULTS[c]
    return df, X, missing

def run_model(X: pd.DataFrame):
    proba = PIPE.predict_proba(X[FEATURES])
    idx = proba.argmax(axis=1)
    return idx, proba
```

Figure 13: application.py: cleaning incoming data and scoring

The function `explain_row` (Figure 14) applies the contribution formula of Section 8.10 to the imputed and standardised row.

application.py – per-prediction explanation

```python
def explain_row(X: pd.DataFrame, pred_idx: int, top=6):
    """Per-feature contribution for the predicted class (works for the linear model)."""
    try:
        z = PIPE.named_steps["scale"].transform(PIPE.named_steps["imp"].transform(X[FEATURES]))[0]
        contrib = z * PIPE.named_steps["clf"].coef_[pred_idx]
        order = np.argsort(-np.abs(contrib))[:top]
        return [{"feature": FEATURES[i], "value": float(X[FEATURES].iloc[0, i]),
                 "impact": float(contrib[i])} for i in order]
    except Exception:
        return []
```

Figure 14: application.py: per-prediction explanation

The single-row endpoint (Figure 15) accepts a JSON object, validates that at least one field matches a model feature, and returns the label, the confidence, the probabilities, the top factors and the number of features that were filled with defaults.

application.py – /api/predict endpoint

```python
@application.route("/api/predict", methods=["POST"])
def predict_one():
    payload = request.get_json(silent=True) or {}
    values = payload.get("values", payload)
    if not isinstance(values, dict) or not values:
        return jsonify(error="Send a JSON object of feature values."), 400
    row = pd.DataFrame([values])
    _, X, missing = clean_frame(row)
    known = [k for k in values if k in FEATURES or k in ("time_stamp",)]
    if not known:
        return jsonify(error="None of the supplied fields match the model's features."), 400
    idx, proba = run_model(X)
    i = int(idx[0])
    code = CODES[i]
    return jsonify(
        code=code, label=LABELS[code], confidence=round(float(proba[0, i]), 4),
        probabilities=pack_probs(proba[0]),
        factors=explain_row(X, i),
        filled_defaults=len(missing), total_features=len(FEATURES),
    )
```

Figure 15: application.py: the /api/predict endpoint

The file endpoint (Figure 16) reads a CSV or Excel file, rejects other types, and applies the same cleaning and scoring as for a single row.

application.py – /api/predict_file endpoint (part)

```python
@application.route("/api/predict_file", methods=["POST"])
def predict_file():
    f = request.files.get("file")
    if f is None or not f.filename:
        return jsonify(error="No file uploaded."), 400
    name = f.filename.lower()
    try:
        if name.endswith(".csv"):
            raw = pd.read_csv(f, nrows=MAX_ROWS + 5)
        elif name.endswith((".xlsx", ".xls")):
            raw = pd.read_excel(f, nrows=MAX_ROWS + 5)
        else:
            return jsonify(error="Please upload a .csv or .xlsx file."), 400
    except Exception as exc:
        return jsonify(error=f"Could not read the file: {exc}"), 400

    df, X, missing = clean_frame(raw)
    if len(df) == 0:
        return jsonify(error="The file has no usable rows."), 400
    if len(missing) == len(FEATURES):
        return jsonify(error="No sensor columns from the model were found. "
                             "Check that the column names match the dataset."), 400
    truncated = len(df) > MAX_ROWS
    df, X = df.iloc[:MAX_ROWS], X.iloc[:MAX_ROWS]
    ...
```

Figure 16: application.py: the start of the /api/predict_file endpoint

9.5 Front end

In the browser, `collectValues` gathers the edited sensor fields, and `parsePasted` accepts either a JSON object or a CSV header with one value line (Figure 17).

static/js/app.js – collecting inputs and parsing a pasted row

```javascript
function collectValues() {
  const out = {};
  Object.entries(inputs).forEach(([k, el]) => { const v = parseFloat(el.value); if (!Number.isNaN(v)) out[k] = v; });
  return out;
}

function parsePasted(text) {
  text = text.trim(); if (!text) throw new Error("Paste a row first.");
  if (text[0] === "{" || text[0] === "[") {
    let o = JSON.parse(text); if (Array.isArray(o)) o = o[0]; return o;
  }
  const lines = text.split(/\r?\n/).filter(Boolean);
  if (lines.length < 2) throw new Error("For CSV paste two lines: a header line and a value line.");
  const split = l => l.split(/[,\t;]/).map(s => s.trim());
  const h = split(lines[0]), v = split(lines[1]), o = {};
  h.forEach((k, i) => { const n = Number(v[i]); o[k] = v[i] !== "" && !Number.isNaN(n) ? n : v[i]; });
  return o;
}
```

Figure 17: app.js: collecting input and parsing a pasted row

9.6 Deployment files

The files in Figure 18 and Figure 19 are all that AWS needs: the process definition, the nginx upload limit, the pinned dependencies and, for container-based hosting, the Dockerfile.

deployment files

```
# ---- Procfile ----
web: gunicorn application:application --bind 0.0.0.0:8000 --workers 2 --timeout 120

# ---- .platform/nginx/conf.d/upload.conf ----
# Elastic Beanstalk's nginx defaults to a 1 MB request limit — raise it so dataset uploads work
client_max_body_size 50M;
proxy_read_timeout 120s;

# ---- requirements.txt ----
Flask==3.1.3
gunicorn==23.0.0
joblib==1.5.3
numpy==2.4.4
pandas==3.0.2
scikit-learn==1.8.0
openpyxl==3.1.5
```

Figure 18: Procfile, nginx configuration and requirements

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY application.py .
COPY model ./model
COPY templates ./templates
COPY static ./static
COPY sample_data ./sample_data
EXPOSE 8080
CMD ["gunicorn", "application:application", "--bind", "0.0.0.0:8080", "--workers", "2", "--timeout", "120"]
```

Figure 19: Dockerfile for container-based deployment

10. Dataset Overview

10.1 Structure of the dataset

The dataset contains the ten-minute SCADA records of two turbines. Table 6 summarises the two machines and Table 7 the class distribution. Turbine 21 was observed for almost three years and contributes about two thirds of the records.

Table 6: Records per turbine

Turbine (asset_id)	Records	Period	Status 3	Status 5
11	54,067	4 Apr 2019 to 15 Apr 2020	429 (0.8 %)	11,661 (21.6 %)
21	106,570	14 Aug 2020 to 24 May 2023	2,933 (2.8 %)	19,085 (17.9 %)
Total	160,637		3,362	30,746

Table 7: Distribution of the target

Status code	Records	Share	In hold-out set
0	126,529	78.8 %	5,665
5	30,746	19.1 %	696
3	3,362	2.1 %	144
Total	160,637	100 %	6,505

The columns of the table fall into the groups of Table 8.

Table 8: Column groups of the raw file

Group	Count	Content
Identifiers and flags	4	`time_stamp`, `asset_id`, `id`, `train_test`
Target	1	`status_type_id` (0, 3, 5)
Average sensors (`*_avg`)	46	ten-minute averages, for example `wind_speed_3_avg`, `power_29_avg`, `sensor_0_avg`
Maximum, minimum and standard-deviation sensors	27	9 each of `*_max`, `*_min`, `*_std`
Sensors without suffix	8	for example `sensor_44`, `sensor_47`

Wind speed has a mean of 6.2 m/s with values from 0.4 to 26.6 m/s. The power column is normalised, with values between 0 and 0.976. After cleaning there are no missing values apart from one in `sensor_14_avg` and one in `sensor_22_avg`.

10.2 Class balance and the two turbines

Figure 20 shows the imbalance. Status 0 dominates, status 5 is a substantial minority and status 3 is rare. The right-hand panel shows that both turbines follow the same pattern, but status 3 is proportionally more frequent on turbine 21 (2.8 % of its records against 0.8 % for turbine 11), so the turbine identifier carries a small amount of information.

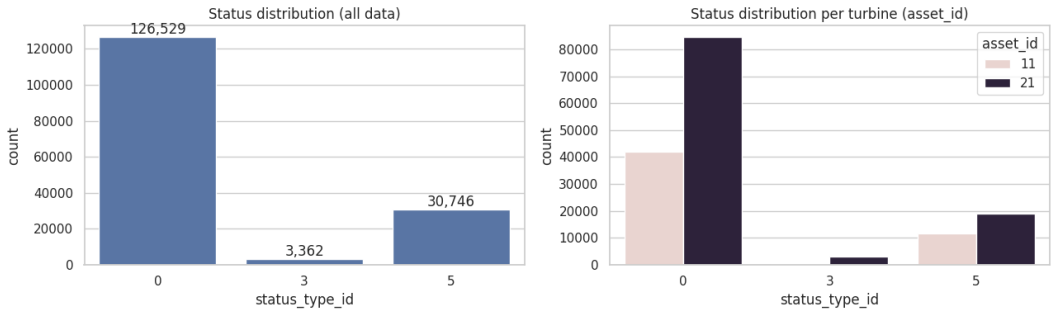


Figure 20: Class distribution, overall and per turbine

10.3 Behaviour over time

Figure 21 gives the monthly share of each status. For both turbines status 0 stays between roughly 70 % and 90 % of the records in most months, with status 5 making up most of the remainder. Status 3 appears as small bursts in a few months, which is what one expects of service visits that are scheduled in blocks. The absence of a strong trend is good news for modelling: the relationship between sensors and status does not appear to drift quickly during the observation period.

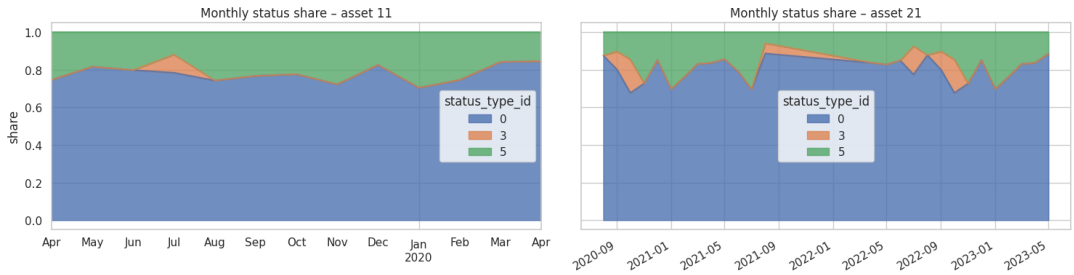


Figure 21: Monthly share of each status for the two turbines

10.4 The power curve

The scatter plot in Figure 22 reproduces the S-shaped power curve described in Chapter 3. Power starts to rise at about 3 to 4 m/s, increases steeply between about 5 and 11 m/s, and then stays at its maximum. Records of status 5 are concentrated at the bottom left, where the wind is too weak to produce power. Quantitatively, **50.1 % of all records with a wind speed below 4 m/s have status 5, against only 2.9 % of those at 8 m/s or more**. Status 5 is therefore, to a large extent, a low-wind condition, which explains why every model identifies it almost perfectly (Chapter 12).

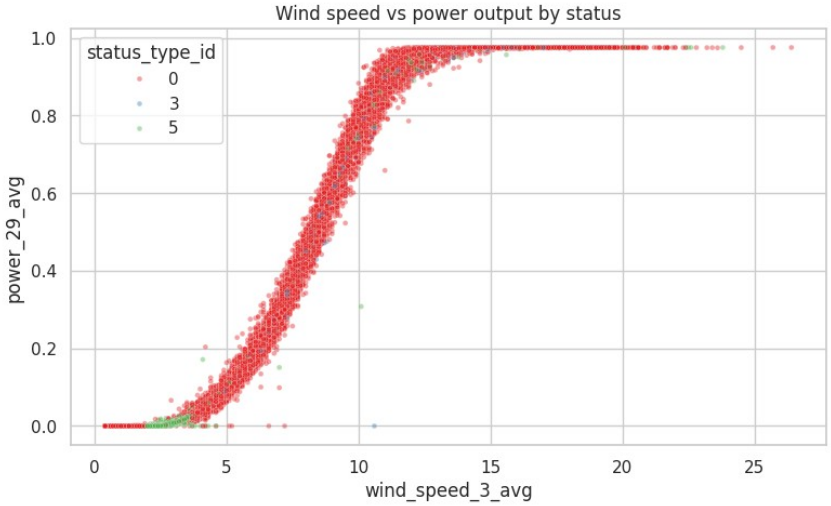


Figure 22: Wind speed against power output, coloured by status

10.5 Sensors by status

The box-plots in Figure 23 and the density plots in Figure 24 compare four sensors across the statuses, and Table 9 lists their averages.

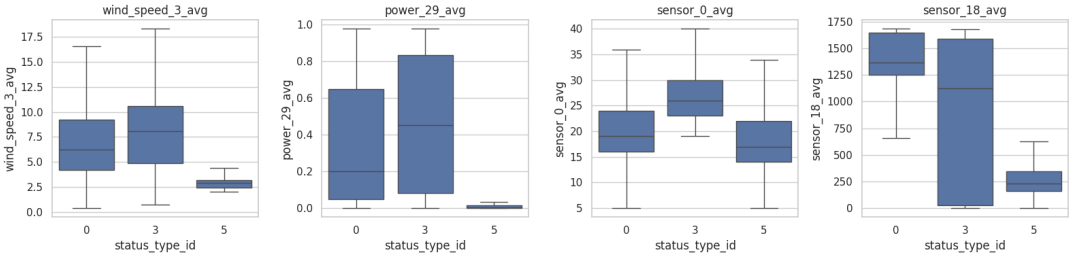


Figure 23: Box-plots of four key sensors by status

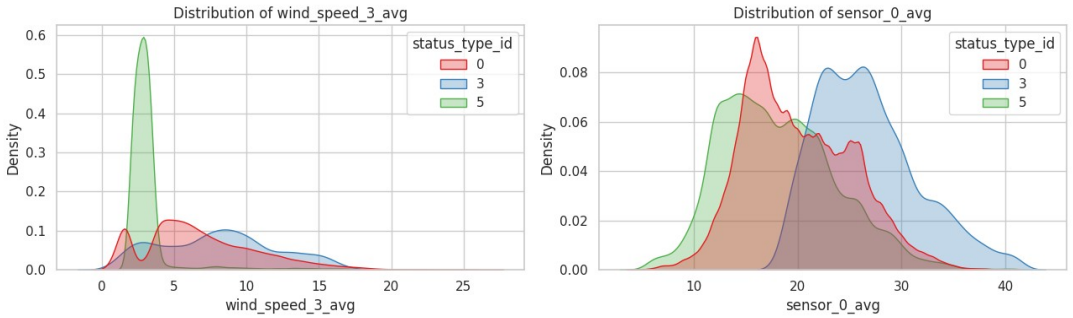


Figure 24: Density of wind speed and sensor_0_avg by status

Table 9: Mean value of selected sensors by status

Sensor	Status 0	Status 3	Status 5
wind_speed_3_avg (m/s)	6.87	8.06	3.30
power_29_avg (normalised)	0.349	0.465	0.047
sensor_0_avg	20.0	26.7	18.1
sensor_18_avg	1,232.8	809.4	327.3
reactive_power_27_avg	0.300	0.192	0.006
sensor_5_avg	4.9	32.9	26.3

Three patterns stand out. First, status 5 differs from the rest in nearly every sensor: low wind, almost zero power and a low value of `sensor_18_avg`. Second, status 3 occurs at *higher* wind speeds than normal operation (mean 8.1 m/s against 6.9 m/s) while `sensor_0_avg` is higher than in the other two classes, which suggests that service stops are not tied to calm weather. Third, `sensor_5_avg` has a very low mean in status 0 and a high mean in statuses 3 and 5, so it separates normal from non-normal operation well, but it does not separate status 3 from status 5. Status 3 overlaps strongly with status 0 in most individual sensors, which is the first indication that it will be the hardest class to predict.

10.6 Time of day

Figure 25 shows that the share of status 5 is highest during the night and early morning (about 23 to 26 %) and drops to about 8 % in the afternoon (hours 15 to 17), while status 3 stays flat at about 2 %. The pattern is consistent with the daily cycle of the wind, but the data does not prove the cause. It does justify keeping `hour` as a feature.

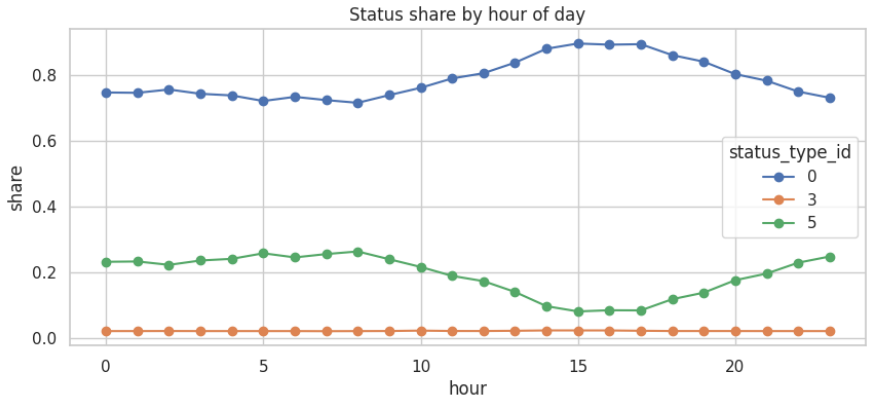


Figure 25: Share of each status by hour of day

10.7 Correlations

Figure 26 ranks the features by the absolute correlation with the encoded target. The largest value is about 0.61 (`sensor_18_min`), followed by `reactive_power_27_avg`, `reactive_power_27_max` and `sensor_18_avg`. Because the target is categorical, this

correlation is only a rough guide to relevance. The heat-map in Figure 27 shows that many of the top features are strongly correlated with one another, for example the minimum, average and maximum of `sensor_18` (0.89 to 0.97) or `sensor_8_avg` and `sensor_11_avg` (0.94). This is the redundancy that the 0.98 correlation filter of Section 8.4 partly removes; pairs below that threshold were kept because they still differ enough to matter, and the regularised models can cope with the remaining correlation.

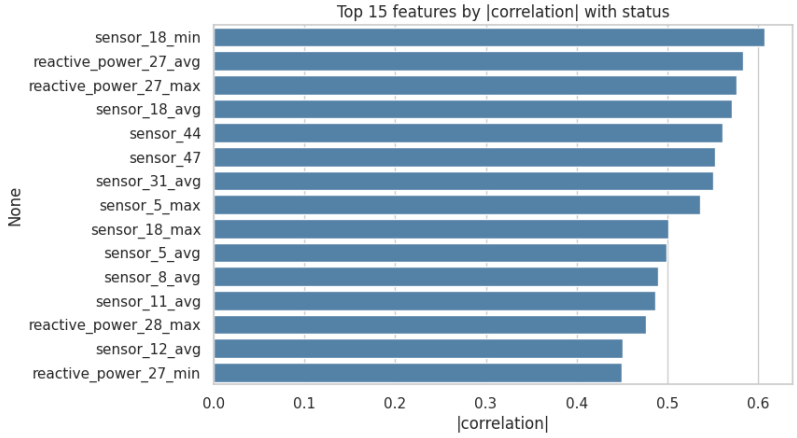


Figure 26: The 15 features most correlated with the status

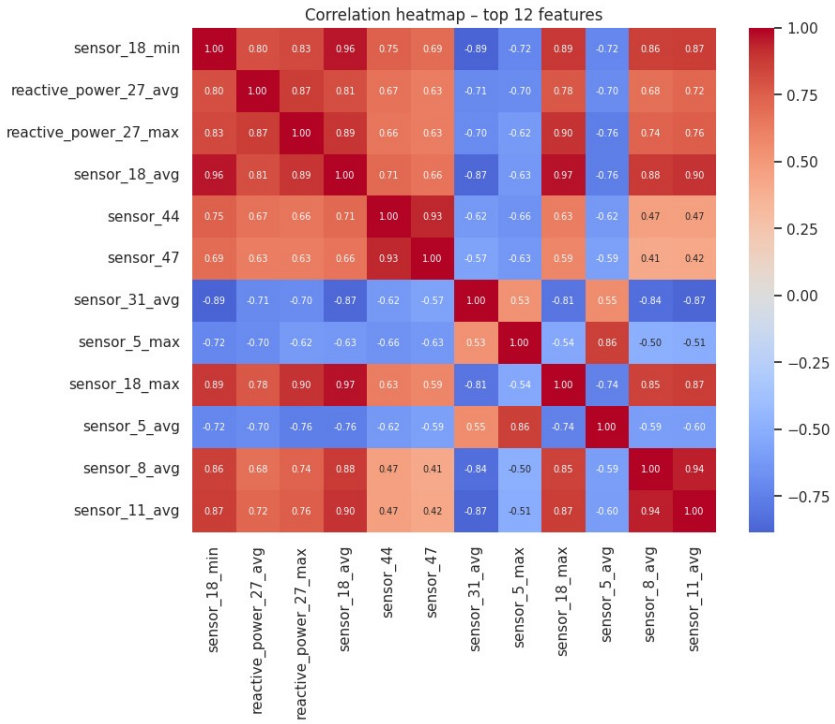


Figure 27: Correlation heat-map of the 12 most relevant features

10.8 What the exploration tells us

The analysis supports three expectations that are tested in the modelling chapter. Status 5 should be easy, because wind and power separate it clearly. Status 0 versus the rest should

be moderately easy through sensors such as `sensor_5_avg` and `sensor_18_avg`. Status 3 should be hard, because it is rare and overlaps with normal operation in most sensors.

11. Input Screenshots

This chapter documents the ways in which a user can give data to the application. All screenshots were taken from the running application.

11.1 Landing page

The landing page (Figure 28) introduces the project with an animated wind-farm scene, a short description and four live figures read from the model file: the number of training records, the number of features, the number of classes and the hold-out macro-F1. The navigation bar leads to the prediction tool, the explanation of the workflow, the model comparison and the deployment overview.

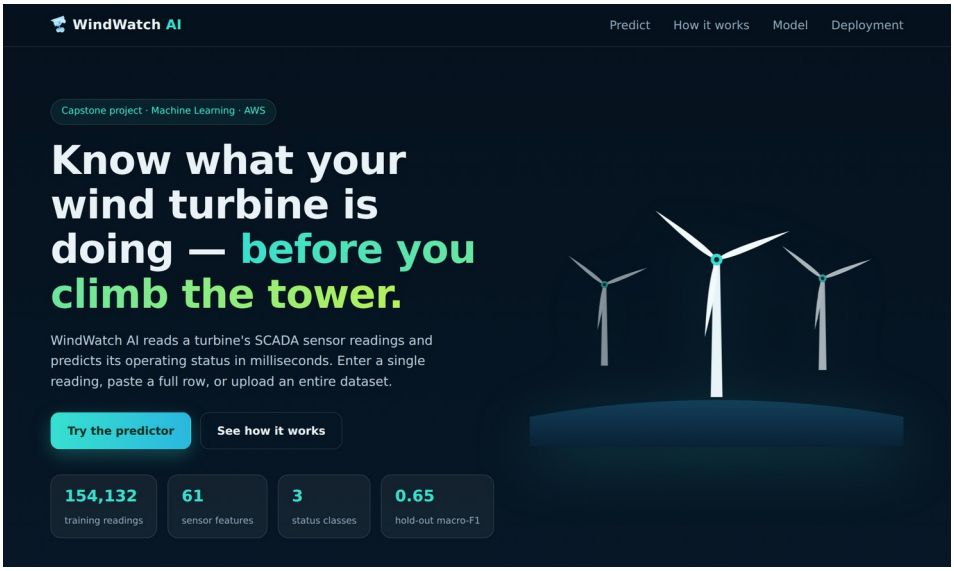


Figure 28: Landing page of WindWatch AI

11.2 Input mode 1: a single reading, sensor by sensor

The first tab (Figure 29) shows the ten most influential sensors as editable fields, already filled with their typical (median) training values. A drop-down offers six real rows taken from the hold-out set (two for each status), and a button restores the typical values. Fields that the user changes are highlighted. A link expands the form to all 61 features (Figure 30). Any field that is left empty is filled automatically.

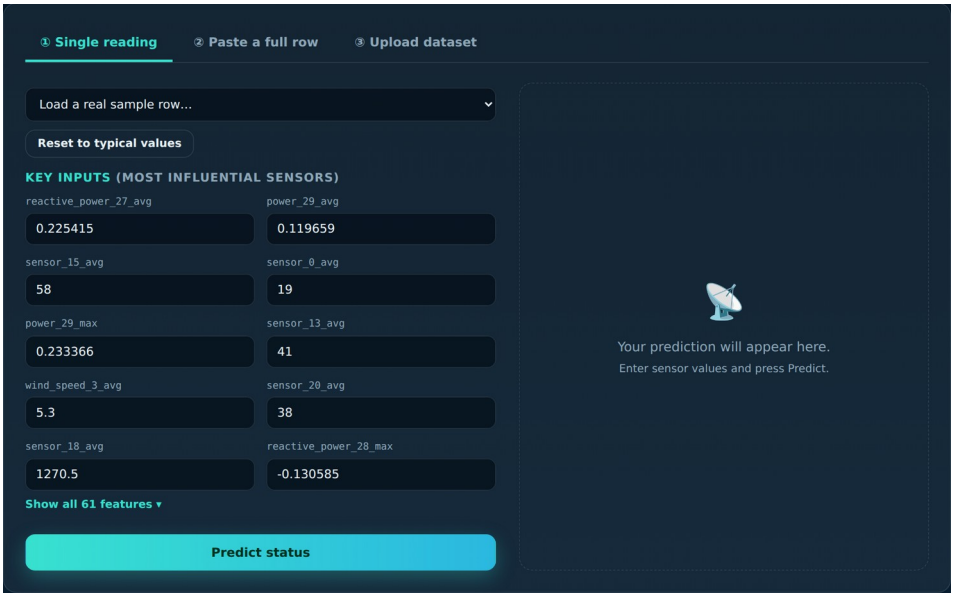


Figure 29: Single reading: key sensors, sample selector and reset button

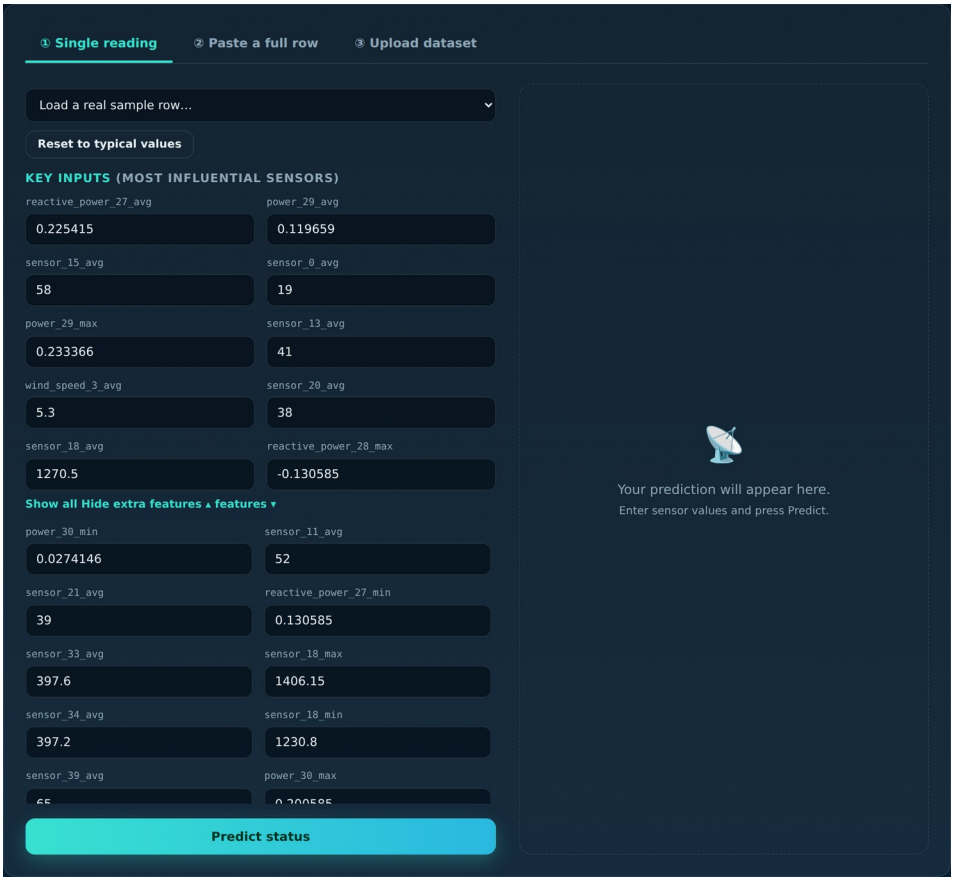


Figure 30: Single reading with all 61 features expanded

11.3 Input mode 2: pasting a complete row

The second tab (Figure 31) accepts a complete row either as a JSON object or as CSV text (a header line followed by one line of values). The button "Insert example row" fills the box with a real row. Columns that are not part of the pasted text are filled with typical values, so a row with only a few columns is also accepted.

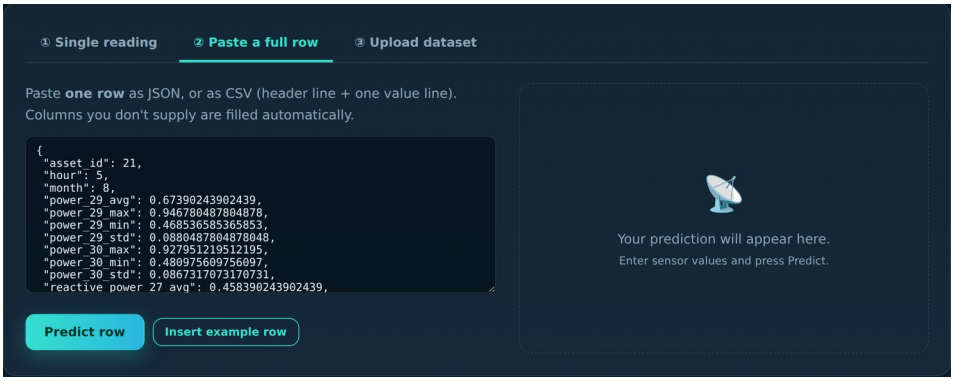


Figure 31: Paste-a-row input containing a complete JSON row

11.4 Input mode 3: uploading a dataset

The third tab (Figure 32) takes a CSV or Excel file by drag-and-drop or by browsing. The file shown here is the 300-row demo file that can be downloaded from the application. It contains the raw column names of the dataset, including `time_stamp`, `asset_id` and the true `status_type_id`, so that the application can also report how well its predictions match the true statuses.

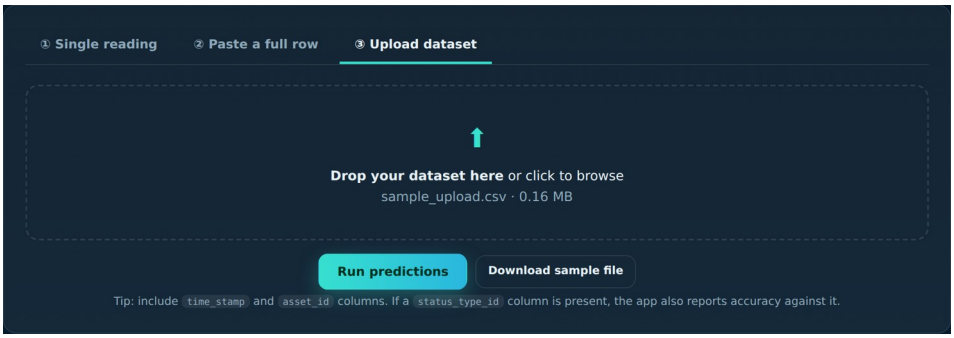


Figure 32: Dataset upload with the demo file selected

11.5 Mobile layout

The page adapts to narrow screens (Figure 33); the form fields stack in one column and the navigation remains available.

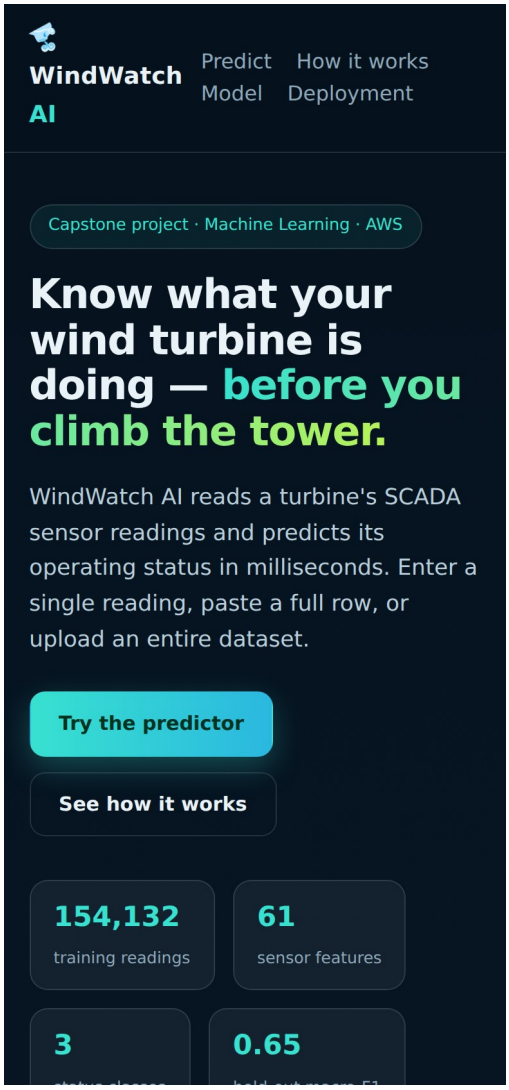


Figure 33: The landing page on a phone-sized screen

12. Output Screenshots and Results

12.1 Output of a single prediction

For every reading the application shows the predicted status with a ring that displays the confidence, the probability of each class as horizontal bars, and the six sensors that influenced the decision most. Figure 34, Figure 35 and Figure 36 show one real hold-out record of each status. The record of status 0 is predicted with 99.8 % confidence; the record of status 3 is predicted correctly with 73.1 % (26.9 % went to normal operation); and the record of status 5 is predicted correctly with 69.2 %, with 29.2 % assigned to normal operation. The probabilities are therefore informative: the less certain the model is, the more the bars are spread.

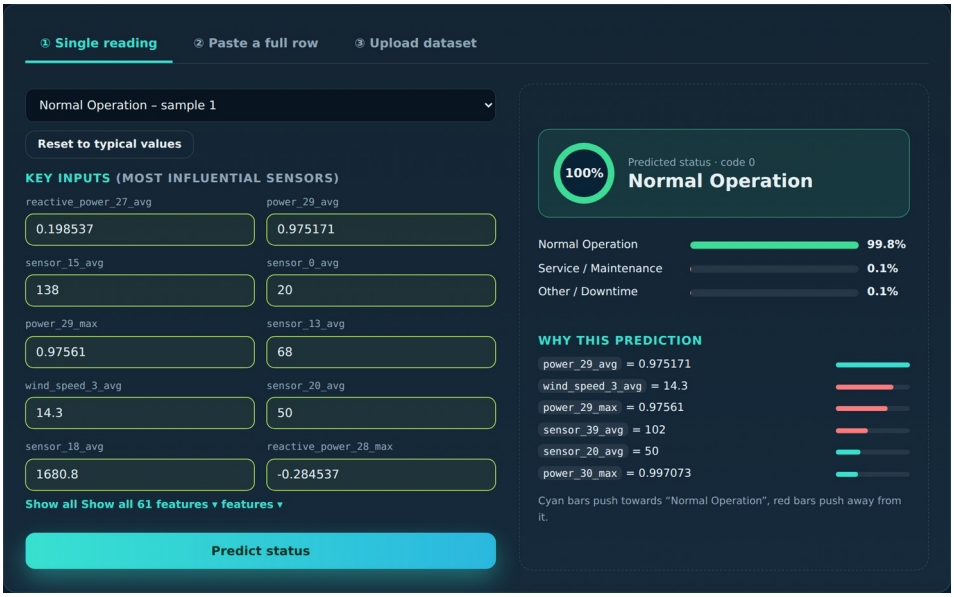


Figure 34: Prediction for a real record of status 0

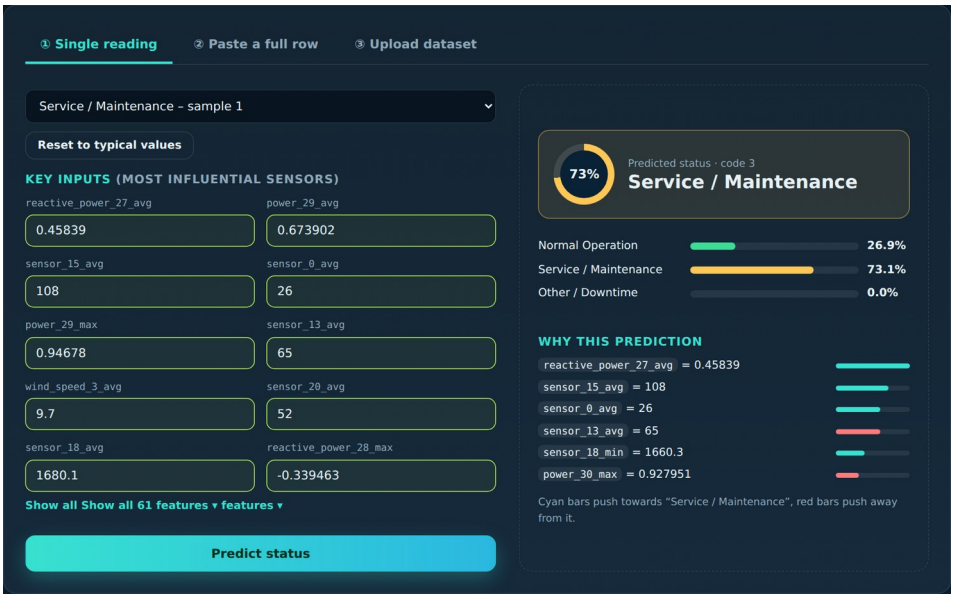


Figure 35: Prediction for a real record of status 3

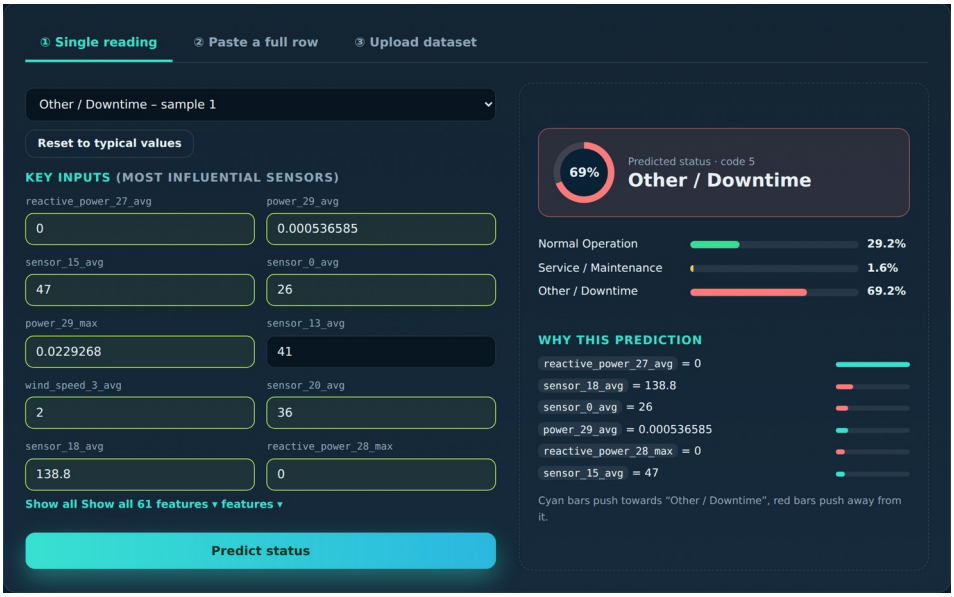


Figure 36: Prediction for a real record of status 5

12.2 Output for a pasted row

Figure 37 shows the result for the pasted JSON row of the previous chapter. Figure 38 shows a deliberately short CSV row containing only five columns (wind speed 9.5 m/s, power 0.8, turbine 21, hour 14, month 6): the model fills the remaining 56 features with typical values and predicts normal operation with 97 % confidence, as expected for a windy afternoon with high power output.



Figure 37: Result for a complete pasted row

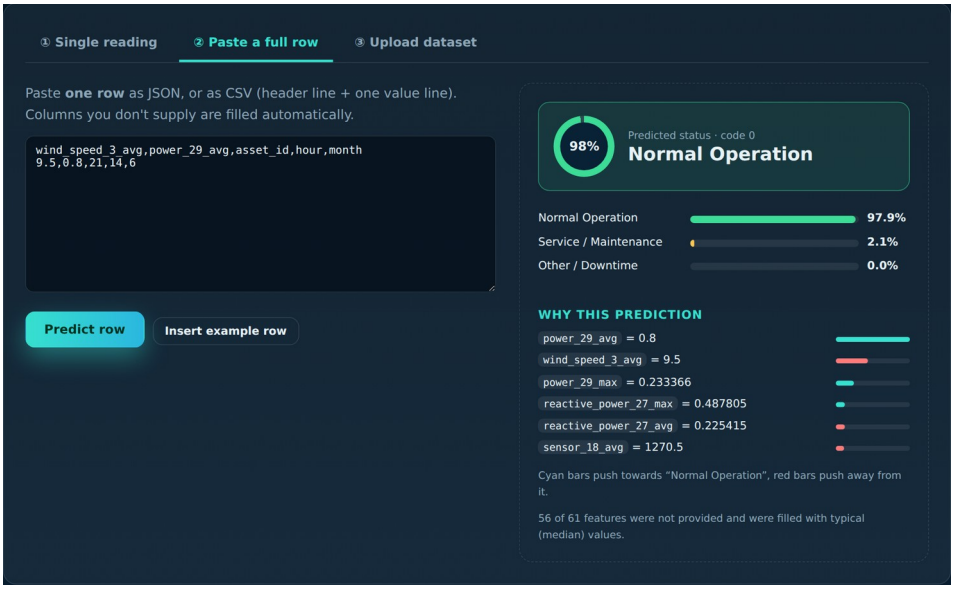


Figure 38: Result for a short CSV row (5 columns only)

12.3 Output for an uploaded dataset

Figure 39 shows the result of scoring the 300-row demo file. The application reports the number of rows, the predicted share of each status (41.3 % normal, 17.3 % service and 41.3 % other), a stacked bar of the same shares, and, because the file contains the true status, an accuracy of 86.7 % and a macro-F1 of 0.833. The table lists the first 100 predictions with the confidence and a tick or cross against the true status, and the whole result can be downloaded as a CSV file. The demo file is deliberately more balanced than the real data (120 normal, 60 service, 120 other rows), which is why its scores are higher than the hold-out macro-F1 of 0.654 and should not be read as the expected performance.



Figure 39: Predictions for an uploaded dataset

12.4 Output through the API

The same functionality is available to other programs (Figure 40). The request sends five sensor values; the response contains the class code, its label, the confidence, the probabilities for the three codes, the top contributing factors and the number of features that had to be filled with defaults.

API request / response (JSON)

```
POST /api/predict
{"values": {"wind_speed_3_avg": 8.4, "power_29_avg": 0.62, "asset_id": 21, "hour": 14, "month": 6}}

--- response ---
{
  "code": 0,
  "confidence": 0.9698,
  "factors": [
    {
      "feature": "power_29_avg",
      "impact": 2.0023063843655193,
      "value": 0.62
    },
    {
      "feature": "wind_speed_3_avg",
      "impact": -0.8900141463747365,
      "value": 8.4
    },
    {
      "feature": "power_29_max",
      "impact": 0.7644676819508385,
      "value": 0.233365853658536
    },
    {
      "feature": "reactive_power_27_max",
      "impact": 0.38158196985417797,
      "value": 0.48780487804878
    },
    {
      "feature": "reactive_power_27_avg",
      "impact": -0.3667621582174487,
      "value": 0.225414634146341
    },
    {
      "feature": "sensor_18_avg",
      "impact": -0.3454806852500704,
      "value": 1270.5
    }
  ],
  "filled_defaults": 56,
  "label": "Normal Operation",
  "probabilities": {
    "0": 0.9698,
    "3": 0.0302,
    "5": 0.0
  },
  "total_features": 61
}
```

Figure 40: Request and response of the /api/predict endpoint

12.5 Model comparison on the hold-out set

All four models were evaluated on the 6,505 hold-out records (5,665 of status 0, 696 of status 5 and 144 of status 3). Table 10 gives the results and Figure 41 shows them graphically.

Table 10: Hold-out results of the four models

Model	Accuracy	Macro precision	Macro recall	Macro-F1	Weighted F1	Train macro-F1
Logistic Regression	0.862	0.607	0.818	0.654	0.889	0.635
Decision Tree	0.960	0.669	0.664	0.640	0.950	0.824
Random Forest	0.963	0.612	0.661	0.635	0.952	0.999
Hist Gradient Boosting	0.959	0.603	0.659	0.629	0.949	0.951

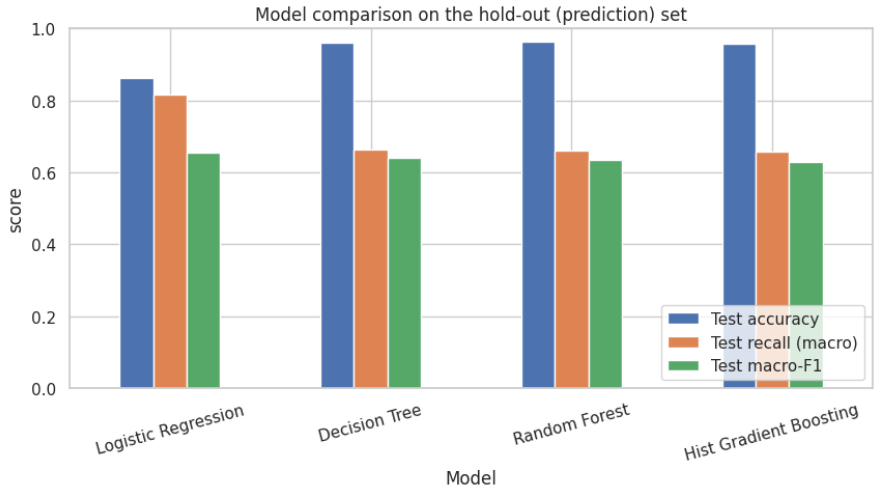


Figure 41: Accuracy, macro-recall and macro-F1 of the four models

The ranking depends on the metric. By accuracy and weighted F1 the three tree-based models are clearly better, but their macro-F1 is lower. By macro-F1 and macro-recall Logistic Regression is first. The difference in macro-F1 between the best and the worst model is only 0.025, so the ranking by this metric alone is not decisive; the per-class results explain why Logistic Regression is nevertheless preferable.

12.6 Per-class results

Table 11: Precision and recall of each model for each status

Model	Recall 0	Recall 3	Recall 5	Precision 3	Precision 5	Status 3 found
Logistic Regression	0.852	0.604	0.997	0.142	0.691	87 of 144
Decision Tree	0.979	0.014	1.000	0.182	0.846	2 of 144
Random Forest	0.983	0.000	1.000	0.000	0.860	0 of 144
Hist Gradient Boosting	0.978	0.000	1.000	0.000	0.833	0 of 144

The table contains the central finding of the project. All four models identify status 5 almost perfectly (recall 0.997 to 1.000), in line with the exploration. For status 3, however, the tree-based models fail: the Random Forest and the Gradient Boosting model never predict it, and the Decision Tree finds only 2 of the 144 records. Logistic Regression finds 87 of 144 (60.4 %). The price is paid in the other classes: it labels 527 of the 5,665 normal records as status 3 and 310 as status 5, so its recall for status 0 is 0.852 against about 0.98 for the trees, and its precision for status 3 is only 0.142.

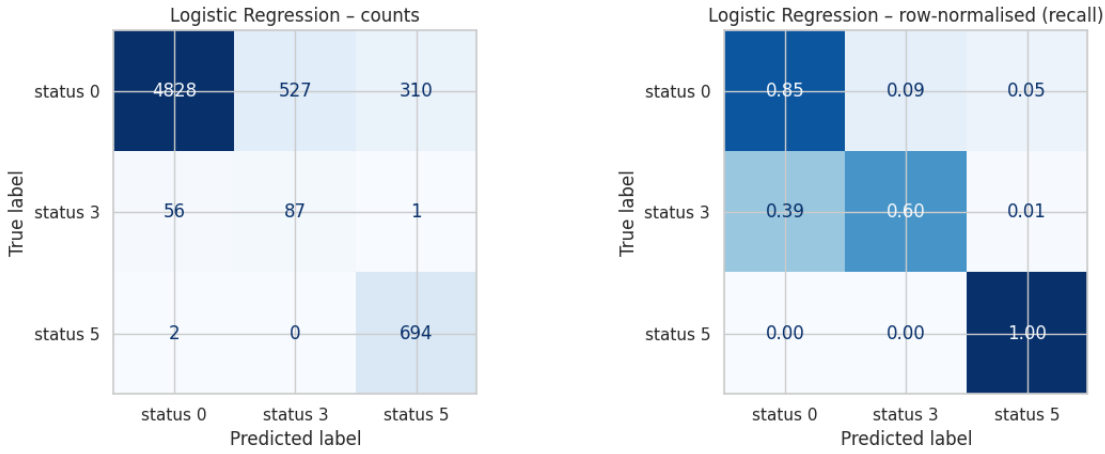


Figure 42: Confusion matrix of Logistic Regression (counts and row-normalised)

12.7 Overfitting check

The last column of Table 10 compares the training macro-F1 with the hold-out value. Logistic Regression scores 0.635 on the training data and 0.654 on the hold-out set, which means that it generalises. Random Forest reaches 0.999 on the training data and 0.635 on the hold-out set, Gradient Boosting falls from 0.951 to 0.629 and the Decision Tree from 0.824 to 0.640. The tree models have learned the training period almost by heart, and their class weighting was not enough to make them recognise status 3 in new data.

12.8 Features that drive the model

Figure 43 ranks the features of the selected model by permutation importance. The most influential feature is `reactive_power_27_avg`, followed by `power_29_avg`, `sensor_15_avg`, `sensor_0_avg` and `power_29_max`; wind speed is also among the top ten. These are electrical output and operating sensors, which agrees with the exploration: the machine's output reflects its state.

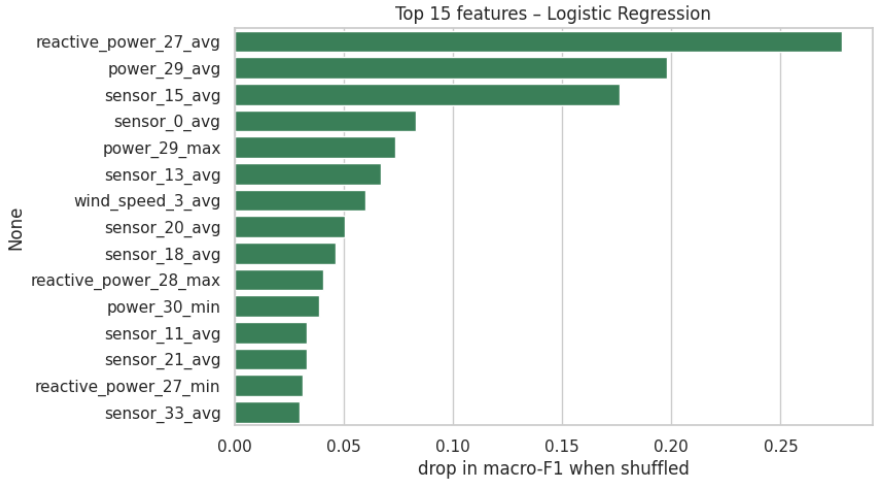


Figure 43: Top 15 features by permutation importance (Logistic Regression)

12.9 Why Logistic Regression was selected

The selection rests on four arguments, in order of weight.

1. **Suitable metric.** With 78.8 % of the records in one class, accuracy rewards ignoring the rare classes. Macro-F1 gives each class the same weight and is the criterion decided before the experiments.
2. **Detection of the rare state.** Logistic Regression has the best macro-recall (0.818) and is the only model that detects a meaningful share of status 3. For a monitoring tool, failing to notice a non-normal state is generally worse than raising an extra alarm.
3. **Generalisation.** Training and hold-out scores are almost equal, whereas the tree models overfit.
4. **Simplicity and transparency.** It trains in seconds, is small (the saved model file is under 10 kB) and provides exact per-prediction explanations.

The limitations deserve the same attention. The advantage in macro-F1 is small. The model produces many false alarms for status 3 (precision 0.142), and its accuracy is lower than that of the trees. Only 144 records of status 3 exist in the hold-out set, so the recall estimates carry a wide margin of uncertainty. If an operator valued a low false-alarm rate above everything else, the Random Forest would be a defensible choice for status 0 and 5, with status 3 handled separately.

13. Conclusion

The project set out to predict the operating status of a wind turbine from its SCADA sensor readings and to make the model usable through a web application. Both parts were completed.

On the data side, the raw Excel file turned out to need careful repair. A header row hidden inside the data had turned every numeric column into text; once this was found and corrected, 160,637 valid records remained, none of them lost. Exploration then showed what the models could be expected to do. Status 5 is largely a low-wind, no-power condition and should be easy. Status 3 is rare, occurs at ordinary wind speeds and overlaps strongly with normal operation, so it should be hard.

On the modelling side, the results confirmed these expectations and added one that is easy to miss. The tree-based models reached 96 % accuracy, which looks excellent, but the per-class results show that they never learned the rare status 3: the Random Forest and the Gradient Boosting model predicted it zero times, and the Decision Tree found 2 of 144 records. A headline accuracy figure would have hidden this entirely. Logistic Regression, with 86 % accuracy, detected 87 of the 144 status-3 records (60.4 %), obtained the best macro-F1 (0.654) and macro-recall (0.818), and showed no sign of overfitting. It was therefore selected as the deployed model.

The choice is not free of cost. The selected model produces many false alarms for status 3 (precision 0.142) and misclassifies 15 % of normal records, and the advantage in macro-F1 over the other models is small. These limitations are stated openly in the application and in this report, because a model that is honest about its weaknesses is more useful to an operator than one that hides them.

On the engineering side, the trained pipeline is stored in a single joblib file and served by a Flask application that accepts a single reading, a pasted row or a dataset, explains each prediction, and can be hosted on AWS in three different ways. The project therefore covers the complete route from a messy spreadsheet to a running service.

14. Summary

- **Problem.** Predict the operating status (0, 3 or 5) of a wind turbine from ten-minute SCADA readings.
- **Data.** 160,637 records from two turbines (April 2019 to May 2023), 81 sensor statistics each. Class shares: status 0, 78.8 %; status 5, 19.1 %; status 3, 2.1 %.

- **Cleaning.** Removed 2 empty rows and 1 repeated header row, converted the columns to numeric types, dropped 2 constant columns and verified that there were no duplicates.
- **Transformation.** Added hour and month, encoded the target, dropped 21 redundant sensors (correlation above 0.98), leaving 61 features, and used the supplied split of 154,132 training and 6,505 hold-out records.
- **Exploration.** Eight charts showed the S-shaped power curve, the concentration of status 5 at low wind (50.1 % of records below 4 m/s), the overlap of status 3 with normal operation and a daily pattern in status 5.
- **Models.** Logistic Regression, Decision Tree, Random Forest and Histogram Gradient Boosting, all with class weighting and a shared pipeline.
- **Results.** Macro-F1: Logistic Regression 0.654, Decision Tree 0.640, Random Forest 0.635, Gradient Boosting 0.629. Accuracy: 0.862, 0.960, 0.963 and 0.959. Status 3 found: 87, 2, 0 and 0 of 144.
- **Selected model.** Logistic Regression, because of the highest macro-F1 and macro-recall, detection of the rare class, no overfitting and transparency.
- **Application.** Flask web app with three input modes (single reading, pasted row, uploaded dataset), probabilities, per-prediction explanations, downloadable results and a health endpoint.
- **Deployment.** Documented for AWS Elastic Beanstalk, EC2 with nginx, and Docker with App Runner.

15. Future Scope

Several extensions would make the system more accurate and more useful.

- **Better handling of status 3.** Oversampling methods such as SMOTE [8], threshold tuning for the rare class, and cost-sensitive boosting may raise its precision without losing recall. A two-stage design (normal against not normal, then status 3 against 5) is another option.
- **Temporal features.** The present model looks at one ten-minute record at a time. Rolling averages, differences and lagged values of the key sensors would let it recognise states that develop over hours.
- **Time-aware validation.** Cross-validation that respects the order of the data, repeated on several periods and on each turbine separately, would give a more reliable estimate than a single hold-out set and would reveal drift.
- **Hyper-parameter search.** The tree models were used with fixed settings; a systematic search, together with probability calibration, might change the ranking.

- **Richer explanations.** SHAP values [12] could provide explanations for non-linear models, so that a stronger model can be deployed without losing transparency.
- **Early fault detection.** Combining the status classifier with normal-behaviour models [4], [5] and evaluating on public benchmark data [6] would move the project from describing the present state to warning about future failures.
- **Monitoring and security in production.** Authentication, a database of past predictions, alerts when the incoming data drifts away from the training data, automated tests and a continuous-integration pipeline would make the service suitable for real operation.
- **Streaming input.** Reading records directly from a message queue or an SCADA export folder, so that predictions appear without manual upload.

16. References

[1] J. Tautz-Weinert and S. J. Watson, "Using SCADA data for wind turbine condition monitoring – a review," *IET Renewable Power Generation*, vol. 11, no. 4, pp. 382–394, 2017.

[2] A. Stetco et al., "Machine learning methods for wind turbine condition monitoring: A review," *Renewable Energy*, vol. 133, pp. 620–635, 2019.

[3] A. Kusiak and W. Li, "The prediction and diagnosis of wind turbine faults," *Renewable Energy*, vol. 36, no. 1, pp. 16–23, 2011.

[4] A. Zaher, S. D. J. McArthur, D. G. Infield and Y. Patel, "Online wind turbine fault detection through automated SCADA data analysis," *Wind Energy*, vol. 12, no. 6, pp. 574–593, 2009.

[5] M. Schlechtingen and I. Ferreira Santos, "Comparative analysis of neural network and regression based condition monitoring approaches for wind turbine fault detection," *Mechanical Systems and Signal Processing*, vol. 25, no. 5, pp. 1849–1875, 2011.

[6] C. Gück, C. M. A. Roelofs and S. Faulstich, "CARE to Compare: A real-world benchmark dataset for early fault detection in wind turbine data," *Data*, vol. 9, no. 12, art. 138, 2024.

[7] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[11] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[12] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York: Springer, 2009.

[15] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.

[16] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.

[17] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

Appendix A: Project Folder Structure

Table 12: Files of the deliverable

File or folder	Purpose
application.py	Flask back end; the entry point used by AWS
templates/index.html	Web page
static/css/style.css, static/js/app.js	Styling and front-end logic
model/wind_turbine_model.joblib	Trained pipeline and metadata
notebook/ wind_turbine_status_prediction.ipynb	Complete data-science workflow with outputs
sample_data/sample_upload.csv	300-row demo file for the upload feature
data/	Location for the original Excel dataset (not bundled, about 82 MB)
requirements.txt	Pinned Python dependencies
Procfile, .platform/, .ebignore	Elastic Beanstalk start command, nginx upload limit, packaging exclusions
Dockerfile, .dockerignore	Container build for App Runner or ECS
docs/	Project synopsis and the AWS deployment guide

Appendix B: Running and Deploying the Project

Run locally. Create a virtual environment, install the requirements with `pip install -r requirements.txt` and start the application with `python application.py`. The page opens at http://127.0.0.1:5000.

Retrain the model. Copy the Excel file into the `data` folder, open the notebook and run all cells. The last cells recreate the joblib file and the demo upload file.

Deploy on Elastic Beanstalk. From the project folder run `eb init`, then `eb create windwatch-env --single`, then `eb open`. Update with `eb deploy` and remove the environment with `eb terminate` to avoid charges.

Other routes. The step-by-step instructions for EC2 with nginx and for Docker with App Runner are in the file `docs/DEPLOYMENT_AWS.md`.

Appendix C: Glossary

Table 13: Terms used in the report

Term	Meaning
SCADA	Supervisory Control and Data Acquisition; the system that logs turbine signals every ten minutes
Status code	Integer written by the turbine controller to describe what the machine was doing in an interval
Macro-F1	Mean of the F1 scores of all classes, giving each class equal weight
Recall	Share of the true members of a class that the model finds
Precision	Share of the model's predictions of a class that are correct
Class weighting	Giving errors on a rare class a larger penalty during training
Overfitting	Learning the training data so closely that performance on new data drops
Pipeline	A chain of preprocessing steps and a model that is fitted and applied as one object
joblib	Python library used to save and load fitted models
Flask	Lightweight Python web framework
Gunicorn	Production web server for Python applications
Elastic Beanstalk	AWS service that deploys and manages a web application from its code
Permutation importance	Drop in a score after a feature is shuffled; a measure of how much the model relies on it